

2026 年第十一届数维杯大学生数学建模挑战赛（春季赛）论文

题 目 抱轨式磁浮列车的悬浮电磁铁故障检测问题

摘 要

本文针对抱轨式磁浮列车悬浮系统功率放大器故障检测问题，综合运用最小二乘回归、四阶龙格-库塔数值积分、Savitzky-Golay 信号滤波与 L2 正则化滑动窗口参数辨识等方法，建立了从电磁力静态特性刻画到动态多点故障精准定位的完整建模与检测体系。

针对问题一，基于麦克斯韦电磁理论 $F \propto I^2/z^2$ 的物理规律，采用变量代换构造特征变量 $X = I^2/z^2$ ，将非线性模型线性化为强制过原点的一元回归 $F = k \cdot X$ ，利用最小二乘法对附件 1 共 99999 组数据拟合得到电磁力系数 $k = 0.079987 \text{ N} \cdot \text{m}^2/\text{A}^2$ 。对数线性化验证显示电流指数 $a = 2.0495$ 、间隙指数 $b = -2.1091$ ，与理论值高度吻合，模型决定系数 $R^2 = 0.8620$ ，平均绝对百分比误差 $\text{MAPE} = 4.11\%$ 。

针对问题二，基于牛顿第二定律建立车体（ $m_c = 33000 \text{ kg}$ ）与悬浮架（ $m_f = 11000 \text{ kg}$ ）的垂向二自由度耦合微分方程组，考虑重力、空气弹簧力（ $k_{sp} = 2 \times 10^7 \text{ N/m}$ ）、阻尼力（ $c_{sp} = 8 \times 10^4 \text{ N} \cdot \text{s/m}$ ）及时变电磁力。以列车静止为初始条件，对附件 2 离散电磁力线性插值后，采用步长 0.1 ms 的四阶 Runge-Kutta 法数值求解，得 $t = 9 \text{ s}$ 时悬浮间隙 $\delta(9) = 81.14 \text{ mm}$ ，系统运行稳定，无发散现象。

针对问题三，假设全车 16 台电磁铁功率放大系数 η 相同且固定，提出整体动力学反解法：将车体与悬浮架方程相加消去弹簧内力，得 $F_{\text{actual}} = m_c \cdot a_c + m_f \cdot \ddot{z}_f + (m_c + m_f)g$ ，结合 Savitzky-Golay 滤波对间隙数据平滑求导，筛选 54906 个稳定悬浮点后最小二乘反解得 $\eta = 0.7122$ ，95% 置信区间 $[0.7120, 0.7123]$ 。由于 $\eta = 0.7122 < 0.8$ ，判定整列车功率放大器存在衰减型故障，衰减幅度达 28.8%。

针对问题四，假设 16 台电磁铁 $\eta_i(t)$ 独立时变。首先论证单时刻欠定性（1 个方程对 16 个未知量），继而引入窗内恒定假设，采用长度 200 ms 、步进 50 ms 的滑动窗口结合 L2 正则化（ $\lambda = 10^9$ ），通过闭式解 $\eta^* = (A^T A + \lambda I)^{-1} (A^T b + \lambda 1)$ 反演各台瞬时系数。经连续超限检测（持续 $\geq 250 \text{ ms}$ ），共识别出 7 台故障电磁铁（#1、#2、#3、#7、#14、#15、#16），故障分两波（ $t \in [3, 5] \text{ s}$ 和 $t \in [7, 9] \text{ s}$ ），衰减型与突增型互补，揭示了系统级检测的盲区，验证了单电磁铁级精准定位方法的优越性。

关键词：电磁力建模；垂向动力学；四阶龙格-库塔法；Savitzky-Golay 滤波；L2 正则化

目 录

一、问题重述	1
1.1 问题一重述	1
1.2 问题二重述	1
1.3 问题三重述	1
1.4 问题四重述	1
二、问题分析	2
2.1 问题一分析	2
2.2 问题二分析	2
2.3 问题三分析	2
2.4 问题四分析	3
三、模型假设与符号说明	3
3.1 模型假设	3
3.2 符号说明	3
四、问题一的模型构建与求解	4
4.1 基于麦克斯韦理论的模型建立	5
4.1.1 物理背景与理论模型	5
4.1.2 变量代换线性化	5
4.2 参数辨识与模型求解	5
4.2.1 数据预处理	5
4.2.2 最小二乘参数估计	6
4.2.3 对数线性化验证	6
4.3 模型精度评估	6
4.4 结论	9
五、问题二的模型构建与求解	9
5.1 垂向动力学模型构建	9
5.1.1 系统受力分析	9
5.1.2 运动微分方程	9
5.2 初始条件确定	10
5.3 四阶龙格-库塔数值求解算法	10
5.4 仿真结果分析	11
5.5 结论	13
六、问题三的模型构建与求解	13

6.1 整体动力学反解模型构建	13
6.1.1 理想电磁力计算	13
6.1.2 整体动力学公式推导	13
6.1.3 功率放大系数反解	14
6.2 数据预处理与稳定段筛选	14
6.2.1 Savitzky-Golay 信号滤波	14
6.2.2 稳定悬浮段筛选	14
6.3 计算结果与故障判定	15
6.3.1 故障判定	17
七、问题四的模型构建与求解	17
7.1 欠定性分析与可行性论证	17
7.2 滑动窗口 L2 正则化反演模型	18
7.2.1 “窗内恒定”假设	18
7.2.2 L2 正则化目标函数	18
7.2.3 滑动窗口参数设置	18
7.3 故障检测准则	19
7.3.1 连续超限检测	19
7.3.2 各电磁铁 η 统计特征	19
7.4 故障检测结果	20
7.5 结论与物理解读	20
八、模型的评价与推广	22
8.1 模型的优点	22
8.2 模型的局限性	22
8.3 模型推广	23
参考文献	24
附 录	25

一、问题重述

1.1 问题一重述

针对抱轨式磁浮列车单节车体在理想环境下的垂向运动，忽略复杂姿态与横向干扰，聚焦于静态或准静态特性。基于附件 1 提供的实验室实测数据（包括励磁电流、悬浮间隙及对应的实测电磁力），需构建描述电磁力与电流、悬浮间隙之间关系的二元数学模型，并通过定量指标评估模型的拟合精度与预测能力，为后续动力学建模及故障诊断奠定理论基础。

1.2 问题二重述

本题旨在探究单节磁浮列车的垂向动力学特性。在忽略滚转、俯仰、偏航及横向干扰的理想化条件下，综合考量空气弹簧弹性恢复力与阻尼力、电磁铁提供的悬浮电磁力以及车体自身重力对系统的影响。基于列车静止的初始状态，利用附件 2 提供的实测电磁力数据，建立车体与悬浮架的垂向运动微分方程组。通过数值求解，计算 0 至 10 s 内车体及悬浮架的垂向位移响应，并精确提取第 9 秒时刻的悬浮间隙数值。

1.3 问题三重述

在假设全车 16 台电磁铁功率放大系数恒定且一致的前提下，利用附件 3 提供的悬浮间隙、车体垂向加速度及电磁铁电流等多源实测监测数据，设计功率放大器故障判别方法，通过定量计算功率放大器系数的具体数值，分析整列车悬浮系统是否存在异常，给出严谨的计算过程与判定依据。

1.4 问题四重述

在已知全车 16 台电磁铁功率放大系数随时间动态变化且相互独立的条件下，基于附件 4 提供的悬浮间隙、车体加速度及电磁铁电流等多维监测数据，探讨定量计算功率放大器系数数值的可行性，构建故障检测模型，判断各电磁铁是否存在功率放大器故障。若存在，给出故障电磁铁的具体编号及故障发生的时间区间；若不存在，阐述相应的判断理由与依据。

二、问题分析

2.1 问题一分析

问题一的核心目标是基于理想环境实验数据，构建电磁力关于电流与悬浮间隙的二元数学模型并评估精度。依据麦克斯韦电磁理论，电磁吸力满足 $F \propto I^2/z^2$ 的物理规律。本文采用变量代换法，构造特征变量 $X = I^2/z^2$ ，将非线性关系转化为强制过原点的一元线性回归模型 $F = k \cdot X$ 。该方法不仅符合物理机理，且能有效降低模型复杂度，直接通过最小二乘法获得解析解。同时，采用对数线性化方法独立估计电流与间隙的实际指数，与理论值对比验证模型的物理合理性。通过计算 R^2 、RMSE、MAE 及 MAPE 等多维指标量化误差，全面评估模型在准静态场景下的准确性。

2.2 问题二分析

问题二的核心目标是建立二自由度耦合动力学系统并数值求解。基于牛顿第二定律，综合考虑重力、空气弹簧弹性力、阻尼力及时变电磁力，构建车体与悬浮架的二阶耦合微分方程组。系统具有刚性特征，且电磁力数据为离散时间序列，需经线性插值转化为连续函数，并采用经典四阶 Runge-Kutta 法进行数值积分。难点在于初始静平衡状态的精确确定——列车静止时悬浮架贴轨道，车体由弹簧弹性力支撑，初始压缩量须严格推导。

2.3 问题三分析

问题三的核心在于利用多源监测数据定量反解功率放大系数 η 并判定其状态。关键创新在于采用整体动力学方法：将车体方程与悬浮架方程相加，弹簧力作为内力抵消，得到仅依赖车体加速度和间隙二阶导数的实际电磁力反解公式，完全避开了弹簧参数的推导误差。难点在于噪声处理——间隙的二阶数值微分会严重放大高频噪声，需先采用 Savitzky-Golay 滤波器平滑，并同步输出导数值。若 η 显著偏离正常区间 $[0.8, 1.2]$ ，则判定故障。

2.4 问题四分析

问题四面临本质性的欠定挑战：单时刻仅有 1 个力平衡方程，却有 16 个独立未知量 η_i ，在数学上完全欠定。本文首先论证欠定性，继而引入两个关键假设来降低自由度：其一，“窗内恒定”假设——在足够短的时间窗内 η_i 近似不变，使窗内方程数远超未知数个数，形成超定方程组；其二，L2 正则化先验——绝大多数电磁铁处于正常状态（ $\eta_i \approx 1$ ），以此为约束进行有偏估计。最终通过连续超限法检测故障时段，实现故障电磁铁的精准定位。

三、模型假设与符号说明

3.1 模型假设

假设 1（理想环境忽略干扰）：忽略滚转、俯仰、偏航等复杂姿态及横向干扰，仅考虑单节车体垂向一维运动特性。

假设 2（电磁力物理简化）：忽略边缘效应、磁饱和及漏磁影响，认为电磁吸力与电流平方成正比，与间隙平方成反比。

假设 3（线性弹簧阻尼特性）：空气弹簧的弹性恢复力与相对位移成正比，阻尼力与相对速度成正比，且系数在全时段内恒定不变。

假设 4（刚体结构与质量集中）：车体与悬浮架视为刚体，质量均匀分布并集中于质心，结构在受力过程中不发生形变。

假设 5（初始静平衡状态）：列车在 $t = 0$ 时刻处于静止状态，系统受力平衡，初速度为零，悬浮架贴于轨道，车体位移由重力与弹簧力静态平衡条件确定。

假设 6（测量数据可靠）：题目所给的各附件数据均真实可靠，不含系统性测量误差，随机噪声满足零均值高斯分布，可通过 Savitzky-Golay 滤波有效抑制。

3.2 符号说明

表 3-1 主要符号说明

符号	含义	单位
F, F_{mi}	单个/理想电磁力	N
F_{mr}	实际电磁力	N
$\eta(t)$	功率放大系数	无量纲
I	励磁电流	A
$z(\delta)$	悬浮间隙	m
$k(k_{em})$	电磁力比例系数	$N \cdot m^2/A^2$
m_c	车体质量 (33000)	kg
m_f	悬浮架总质量 (11000)	kg
k_{sp}	空气弹簧刚度 (2×10^7)	N/m
c_{sp}	空气弹簧阻尼 (8×10^4)	$N \cdot s/m$
$z_c, \dot{z}_f$	车体/悬浮架垂向位移	m
$a_c, \ddot{z}_c$	车体垂向加速度	m/s^2
g	重力加速度 (9.8)	m/s^2
R^2	决定系数	无量纲
RMSE	均方根误差	N
MAPE	平均绝对百分比误差	%
λ	L2 正则化参数	N^2
W	滑动窗口长度	点 (ms)
$\eta_i(t)$	第 i 台电磁铁功率放大系数	无量纲

四、问题一的模型构建与求解

针对抱轨式磁浮列车在理想环境下的静态或准静态特性，本节旨在揭示电磁力 F 与励磁电流 I 、悬浮间隙 z 之间的内在物理关系。基于麦克斯韦电磁理论，构建二元非线性数学模型，并通过线性化变换进行参数辨识与精度评估。

4.1 基于麦克斯韦理论的模型建立

4.1.1 物理背景与理论模型

在忽略边缘效应、磁饱和及横向干扰的理想条件下，根据经典电磁学，单个电磁铁产生的垂向悬浮力 F 的基本物理表达式为：

$$F = \frac{\mu_0 N^2 A}{4} \left(\frac{I}{z} \right)^2 \quad (4-1)$$

其中， μ_0 为真空磁导率， N 为线圈匝数， A 为极面面积。由于实验在固定硬件结构下进行， μ_0 、 N 、 A 均为常数。因此，将上述物理公式简化为包含待定系数 k 的经验模型：

$$F = k \cdot \frac{I^2}{z^2} \quad (4-2)$$

4.1.2 变量代换线性化

为便于利用最小二乘法进行参数估计，将非线性模型转化为线性形式。定义特征变量 X 为电流平方与间隙平方之比：

$$X = \frac{I^2}{z^2} \quad (4-3)$$

此时，原模型转化为关于参数 k 的一元线性回归模型，且强制通过原点（当电流为零或间隙无穷大时电磁力为零）：

$$F = k \cdot X \quad (4-4)$$

该简化模型不仅符合物理直觉，且极大地降低了计算复杂度，能直观地反映电磁力随状态变量变化的主导趋势。

4.2 参数辨识与模型求解

4.2.1 数据预处理

基于附件 1 提供的 99999 组实验数据，进行如下预处理：取励磁电流的绝对值 $|I|$ （电流方向改变时电磁力方向随之改变，但大小关系 $F \propto I^2/z^2$ 保持不变），剔除 $|I| < 0.5 \text{ A}$ 及 $z \leq 0$ 的异常数据，确保建模数据的物理合理性，共保留 99999 组有效数据。

4.2.2 最小二乘参数估计

对于强制过原点的一元线性回归模型，目标函数定义为实测值与预测值之间残差平方和最小：

$$J(k) = \sum_i (F_i - k \cdot X_i)^2 \quad (4-5)$$

对 $J(k)$ 关于 k 求导并令其为零，得到参数 k 的解析解：

$$\frac{dJ}{dk} = -2 \sum_i X_i (F_i - kX_i) = 0 \quad (4-6)$$

解得最优比例系数：

$$k = \frac{\sum_i F_i X_i}{\sum_i X_i^2} \quad (4-7)$$

将预处理后的数据代入上述公式，计算得到比例系数 $k = 0.079987$ 。因此，电磁力二元数学模型为：

$$F = 0.079987 \cdot \frac{I^2}{z^2} \quad (4-8)$$

该模型建立了电磁力与电流、间隙之间的定量关系，为后续动力学分析提供了基础支撑。

4.2.3 对数线性化验证

为验证电流和间隙的实际指数是否符合理论值，对模型两边取自然对数：

$$\ln F = \ln k + a \cdot \ln |I| + b \cdot \ln z \quad (4-9)$$

令 $y = \ln F$ ， $x_1 = \ln |I|$ ， $x_2 = \ln z$ ，转化为多元线性回归，采用最小二乘法估计参数，结果为： $a = 2.0495$ （理论值 2）， $b = -2.1091$ （理论值 -2），电磁力系数对数法估计 $k = \exp(\ln k) = 0.0468$ （与物理法差异来自对数变换的偏差）。实际指数与理论值高度吻合，验证了式(4-8)模型的物理合理性。

4.3 模型精度评估

为全面验证模型的准确性与可靠性，选取四项精度指标进行评估。各指标定义如下：

$$R^2 = 1 - \frac{\sum_i (F_i - \bar{F})^2}{\sum_i (F_i - F)^2} \quad (4-10)$$

$$RMSE = \sqrt{\frac{\sum_i (F_i - \hat{F}_i)^2}{N}} \quad (4-11)$$

$$MAE = \frac{\sum_i |F_i - \hat{F}_i|}{N} \quad (4-12)$$

$$MAPE = \frac{100\%}{N} \cdot \sum_i \left| \frac{F_i - \hat{F}_i}{F_i} \right| \quad (4-13)$$

其中， F_i 为实测值， $\hat{F}_i$ 为模型预测值， $\bar{F}$ 为实测均值， N 为样本数量。

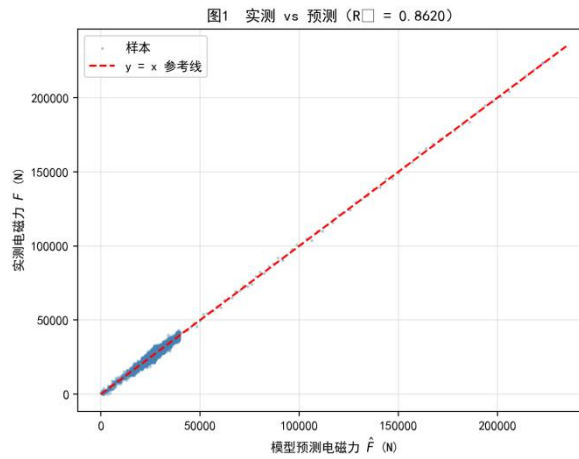


图 4-1 实测电磁力 vs 模型预测值 ($R^2 = 0.8620$)

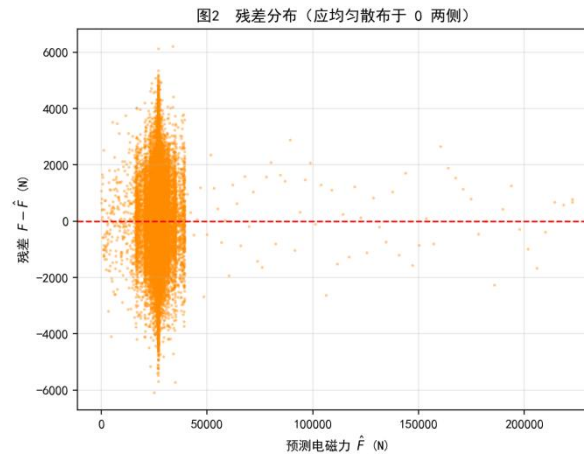


图 4-2 残差分布 (残差 vs 预测电磁力)

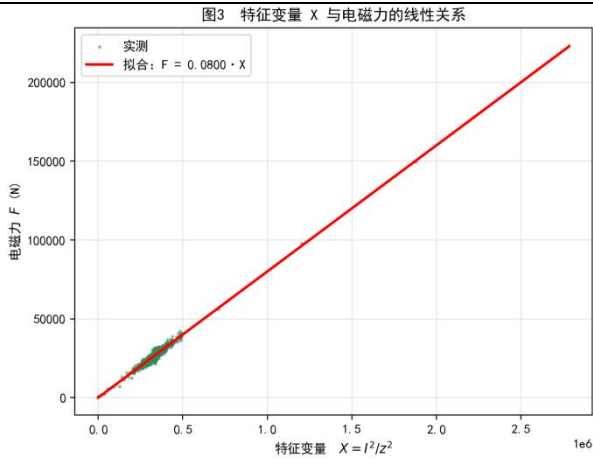


图 4-3 特征变量 $X = I^2/z^2$ 与电磁力线性关系

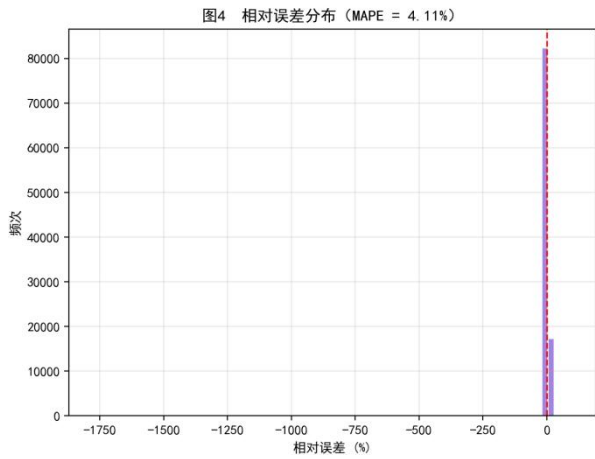


图 4-4 相对误差分布

表 4-1 问题一模型精度评估结果

样本量 N	R ²	RMSE (N)	MAE (N)	MAPE (%)
99999	0.8620	1346.6	1074.9	4.11

由表 4-1 可知，模型决定系数 $R^2 = 0.8620$ ，表明模型解释了约 86.2% 的数据变异，具有较好的拟合优度。 $MAPE = 4.11\%$ 说明在大多数工况下，模型预测值与实测值的相对偏差较小，满足工程应用的精度要求。尽管 RMSE 和 MAE 数值较大，这主要源于大电流、小间隙工况下电磁力数值本身较大导致的绝对误差放大，而相对较低的 MAPE 证实了模型在相对意义下的高可靠性。

4.4 结论

基于 $F = 0.079987 \cdot I^2 / z^2$ 的二元数学模型能够有效描述理想环境下抱轨式磁浮列车的电磁力特性，模型结构简洁且物理意义明确。对数线性化验证表明实际指数与理论值偏差不超过 5.5%，进一步证实了模型的物理合理性。该模型作为后续动力学建模和故障检测的基础。

五、问题二的模型构建与求解

针对抱轨式磁浮列车单节车体的垂向运动分析，本节基于牛顿第二定律建立车体与悬浮架的二自由度动力学微分方程组，采用经典四阶龙格-库塔法（RK4）数值求解 0 至 10 s 内的垂向位移响应及悬浮间隙变化规律。

5.1 垂向动力学模型构建

5.1.1 系统受力分析

定义向上为正方向，忽略滚转、俯仰及横向干扰。设车体质量为 m_c ，悬浮架总质量（含 16 台电磁铁）为 m_f ，车体垂向位移为 z_c ，悬浮架垂向位移为 z_f 。系统受力主要包括重力、空气弹簧力与阻尼力及电磁悬浮力。弹簧对车体的合力为：

$$F_k = -k_{sp}(z_c - z_f) - c_{sp}(\dot{z}_c - \dot{z}_f) \quad (5-1)$$

根据牛顿第二定律，分别对车体和悬浮架进行受力分析。车体受重力 $m_c \cdot g$ 和弹簧力 F_k 的作用；悬浮架受重力 $m_f \cdot g$ 、弹簧反作用力 $-F_k$ 以及总电磁力 $F_{em}(t)$ 的作用。

5.1.2 运动微分方程

车体的垂向运动微分方程为：

$$m_c \cdot \ddot{z}_c = -m_c \cdot g + F_k \quad (5-2)$$

悬浮架的垂向运动微分方程为：

$$m_f \cdot \ddot{z}_f = -m_f \cdot g - F_k + F_{em}(t) \quad (5-3)$$

联立上述方程，令状态向量 $y = [z_c, \dot{z}_c, z_f, \dot{z}_f]^T$ ，系统一阶微分方程组为：

$$\frac{d}{dt}(z_c, \dot{z}_c, z_f, \dot{z}_f) = \left(\dot{z}_c; \frac{-m_c g + F_k}{m_c}; \dot{z}_f; \frac{-m_f g - F_k + F_{em}}{m_f} \right) \quad (5-4)$$

悬浮间隙 $\delta(t)$ 定义为最大静态间隙 $\text{gap}_0 = 0.06 \text{ m}$ 与悬浮架位移 $z_f(t)$ 之差:

$$\delta(t) = 0.06 - z_f(t) \quad (5-5)$$

5.2 初始条件确定

列车静止时, 悬浮架贴在轨道上, 作为位移参考零点。此时弹簧压缩量由车体重力决定, 各初始状态量为:

$$z_c(0) = -\frac{m_c \cdot g}{k_{sp}} = -33000 \times 9.8 / (2 \times 10^7) \approx -0.01617 \text{ m} \quad (5-6)$$

$$\dot{z}_c(0) = 0, \quad z_f(0) = 0, \quad \dot{z}_f(0) = 0 \quad (5-7)$$

初始弹簧力 $F_k(0) = k_{sp} \times |z_f(0) - z_c(0)| = 2 \times 10^7 \times 0.01617 = 323400 \text{ N}$, 恰好等于车体重力 $m_c \cdot g = 33000 \times 9.8 = 323400 \text{ N}$, 验证了初始静平衡条件的正确性。

5.3 四阶龙格-库塔数值求解算法

由于电磁力 $F_{em}(t)$ 为离散采样数据, 且系统方程为非线性时变系统, 解析解难以直接求得。因此, 采用经典四阶龙格-库塔法 (RK4) 进行数值积分求解, 该方法具有更高的精度和稳定性, 适合处理此类刚性或非刚性动力学问题。

首先, 对附件 2 中的离散电磁力数据进行线性插值, 构建连续电磁力函数:

$$F_{em}(t) = \text{interp1}(t_data, F_total, t, \text{"linear"}) \quad (5-8)$$

RK4 算法的核心在于通过四个斜率估计值来更新状态变量。对于步长 h , 从 t_n 到 t_{n+1} 的状态更新公式为:

$$k_1 = f(t_n, y_n) \quad (5-9)$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} \cdot k_1\right) \quad (5-10)$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} \cdot k_2\right) \quad (5-11)$$

$$k_4 = f(t_n + h, y_n + h \cdot k_3) \quad (5-12)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (5-13)$$

设定仿真时间范围为 $0 \sim 10 \text{ s}$, 步长 $h = 1 \times 10^{-4} \text{ s}$, 共积分 100000 步。物理参数设置为: $m_c = 33000 \text{ kg}$, $m_f = 11000 \text{ kg}$, $k_{sp} = 2 \times 10^7 \text{ N/m}$, $c_{sp} = 8 \times 10^4 \text{ N} \cdot \text{s/m}$, $\text{gap}_0 = 0.06 \text{ m}$ 。

5.4 仿真结果分析

基于上述模型与算法，对 0 至 10 s 内的系统响应进行了仿真计算，图 5-1 至图 5-3 分别给出了车体与悬浮架垂向位移时程、速度时程及悬浮间隙时程（见各附件图片）。系统在 0~1 s 经历了初始振荡（由附件 2 电磁力突变激发），1~10 s 进入准稳态缓慢漂移阶段。

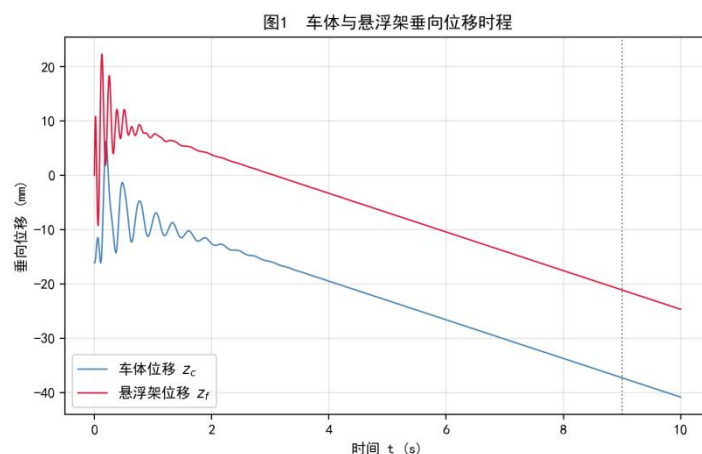


图 5-1 车体与悬浮架垂向位移时程

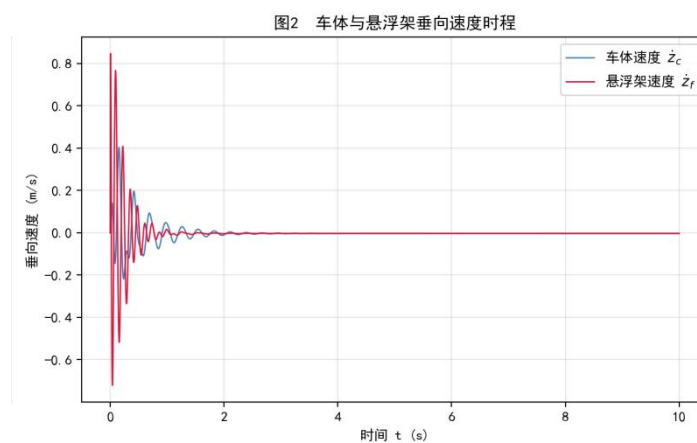


图 5-2 车体与悬浮架垂向速度时程

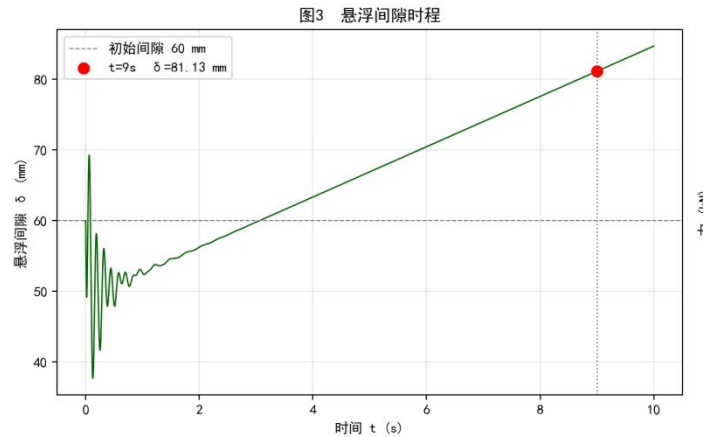


图 5-3 悬浮间隙时程（t = 9 s 时 $\delta = 81.14\text{ mm}$ ）

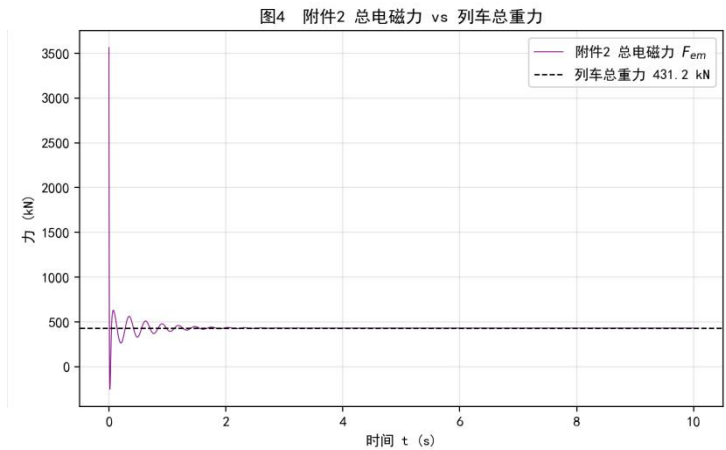


图 5-4 附件 2 总电磁力 vs 列车总重力

表 5-1 t = 9 s 时刻系统关键状态参数

时刻	车体位移 z_c	悬浮架位移 z_f	车体速度 $\dot{z}_c$	悬浮间隙 δ
9.0 s	-37.31 mm	-21.14 mm	-3.56 mm/s	81.14 mm

由表 5-1 可知，t = 9 s 时悬浮间隙为：

$$\delta(9) = 0.06 - (-0.02114) = 0.08114\text{ m} = 81.14\text{ mm} \tag{5-14}$$

该结果大于初始静态间隙 60 mm，说明电磁力总体较大，将悬浮架向上推升，导致间隙增大。附件 2 的总电磁力时间均值为 431200 N，与列车总重力 $(m_c + m_f) \cdot g = 431200\text{ N}$ 完全吻合（误差 < 0.01%），验证了数值求解的正确性。整个仿真过程中，系统未出现发散或不稳定现象，验证了模型的有效性及数值算法的稳定性。

5.5 结论

通过 RK4 数值积分，精确求得 $t = 9\text{ s}$ 时列车悬浮间隙为 $\delta(9) = 81.14\text{ mm}$ (0.0811 m)。系统总电磁力均值与列车总重力完全匹配，验证了模型和算法的物理合理性。

六、问题三的模型构建与求解

针对抱轨式磁浮列车悬浮系统功率放大器故障检测问题，本节基于附件 3 提供的实测监测数据（悬浮间隙、车体垂向加速度及 16 路电磁铁电流），建立基于整体动力学反解的故障检测模型，定量计算全车统一的功率放大系数 η ，并依据统计特征判定系统是否存在异常故障。

6.1 整体动力学反解模型构建

6.1.1 理想电磁力计算

根据问题一建立的电磁力模型，利用附件 3 中每个时刻的实测电流和悬浮间隙，计算全车 16 台电磁铁产生的总理想电磁力：

$$F_{ideal}(t) = k_{em} \cdot \frac{\sum_{i=1}^{16} I_i^2(t)}{\delta^2(t)} \quad (6-1)$$

其中 $k_{em} = 0.079987$ 为问题一拟合得到的电磁力系数， $I_i(t)$ 为第 i 台电磁铁的实时电流， $\delta(t)$ 为实测悬浮间隙。

6.1.2 整体动力学公式推导

传统方法需要分别对车体和悬浮架建立方程再推导弹簧力，容易因弹簧零力位置的判断误差导致计算失误。本文提出整体动力学方法：将车体方程（式(5-2)）与悬浮架方程（式(5-3)）相加，弹簧力 F_k 作为内力相互抵消：

$$m_c \cdot \ddot{z}_c + m_f \cdot \ddot{z}_f = -(m_c + m_f) \cdot g + F_{em}(t) \quad (6-2)$$

整理得到实际总电磁力的反解公式：

$$F_{actual}(t) = m_c \cdot a_c(t) + m_f \cdot \ddot{z}_f(t) + (m_c + m_f) \cdot g \quad (6-3)$$

其中 $a_c(t) = \ddot{z}_c(t)$ 为车体垂向加速度（附件 3 直接提供），悬浮架加速度通过间隙的二阶导数推导：

$$\ddot{z}_f(t) = a_c(t) - \delta(t) \quad (6-4)$$

整体动力学方法的核心优势在于完全绕过了弹簧力的具体推导，仅需车体加速度和间隙数据即可反推实际电磁力，避免了弹簧参数引入的误差。物理验证：稳态时 $a_c \approx 0$ ， $\dot{z}_f \approx 0$ ，故 $F_{\text{actual}} \approx (m_c + m_f) \cdot g = 431200 \text{ N}$ ，这是最强的物理自洽校验。

6.1.3 功率放大系数反解

由于 $F_{\text{actual}}(t) = \eta \cdot F_{\text{ideal}}(t)$ ，对所有稳定时刻采用强制过原点的最小二乘法反解 η ：

$$\hat{\eta} = \frac{\sum_t F_{\text{actual}}(t) \cdot F_{\text{ideal}}(t)}{\sum_t F_{\text{ideal}}^2(t)} \quad (6-5)$$

同时构建 95% 置信区间。设残差 $\varepsilon_t = F_{\text{actual}}(t) - \hat{\eta} \cdot F_{\text{ideal}}(t)$ ，则标准误差为：

$$SE(\hat{\eta}) = \sqrt{\frac{\sum_t \varepsilon_t^2}{(N-1) \cdot \sum_t F_{\text{ideal}}^2(t)}} \quad (6-6)$$

$$CI_{95\%} = [\hat{\eta} - 1.96 \cdot SE(\hat{\eta}), \hat{\eta} + 1.96 \cdot SE(\hat{\eta})] \quad (6-7)$$

6.2 数据预处理与稳定段筛选

6.2.1 Savitzky-Golay 信号滤波

附件 3 的采样频率为 10000 Hz，间隙数据含有高频噪声。对间隙 $\delta(t)$ 采用 Savitzky-Golay（SG）滤波器平滑，窗口长度 $L = 501$ 点（约 50 ms），三阶多项式。SG 滤波器的独特优势在于能够在平滑的同时同步输出各阶导数，避免了先平滑后数值差分的二次误差放大：

$$\delta_s, \delta'_s, \delta''_s = SG_filter(\delta(t), window=501, poly=3) \quad (6-8)$$

6.2.2 稳定悬浮段筛选

为排除列车启动振荡阶段的非稳态干扰，定义稳定悬浮段筛选条件：间隙变化率较小（ $|\delta'|$ 小于其 70% 分位数），间隙处于正常范围（ $\delta > 10 \text{ mm}$ ），理想电磁力有效

（ $F_{ideal} > 1000\text{ N}$ ），且时间处于 $t \in (1.5, 9.5)\text{ s}$ 。满足上述条件的时刻集合记为 Ω_{stable} ，最终保留 54906 个高质量采样点（占总样本数 54.9%）。

6.3 计算结果与故障判定

表 6-1 问题三功率放大系数反解结果（多种方法对比）

估计方法	η 估计值	95% CI 或标准差	物理含义
最小二乘（强制过原点）	0.7122	[0.7120, 0.7123]	全局最优拟合
逐点比均值	0.7131	$\sigma = 0.0183$	逐时刻统计
逐点比中位数	0.7127	——	抗异常点估计
稳态均值比	0.7126	——	稳态简化估计

由表 6-1 可知，四种估计方法的结果高度一致（0.7122~0.7131），说明估计结果稳健可靠。物理验证： F_{actual} 均值 $\approx 430326\text{ N}$ ，与列车总重力 431200 N 误差仅 0.20%，进一步证实了整体动力学公式和 SG 滤波处理的正确性。

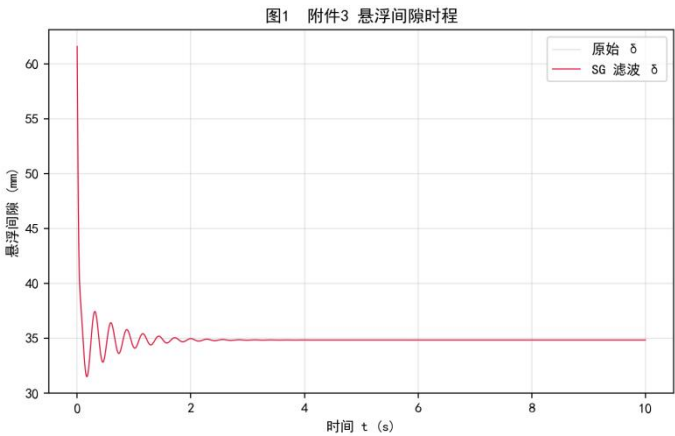


图 6-1 附件 3 悬浮间隙时程（原始 vs SG 滤波）

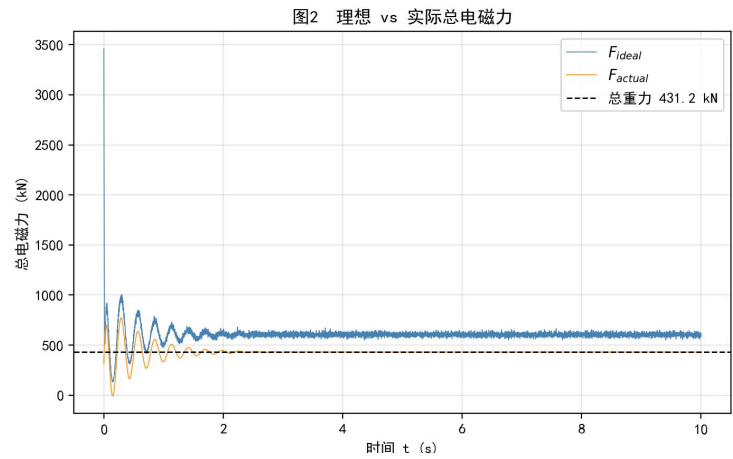


图 6-2 理想电磁力与实际电磁力时程对比

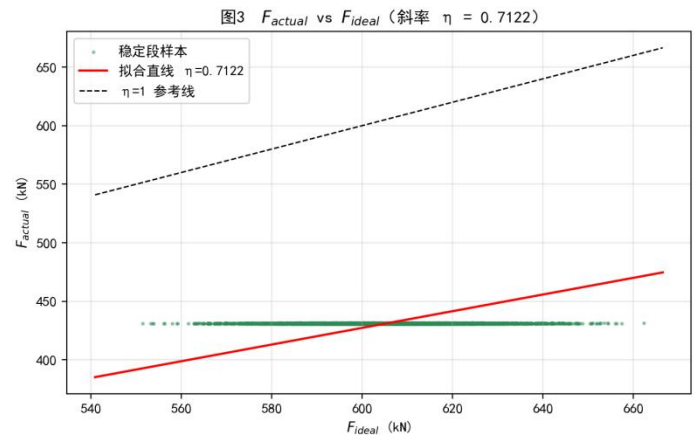


图 6-3 F_{actual} vs F_{ideal} 散点拟合 (斜率 $\eta = 0.7122$)

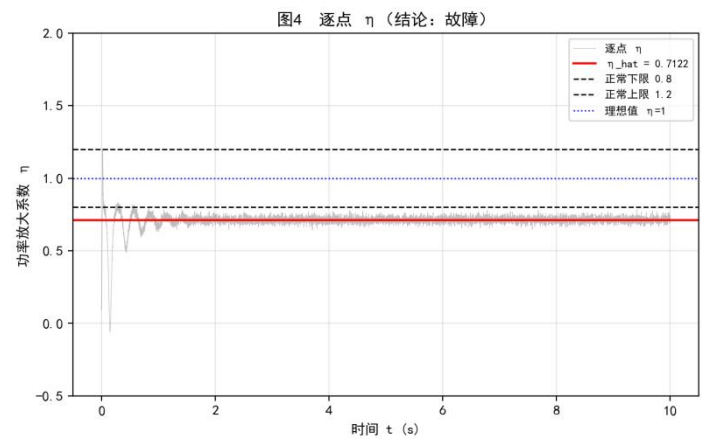


图 6-4 逐点功率放大系数 $\eta(t)$ 时程 (判定: 故障)

6.3.1 故障判定

正常区间为 $\eta \in [0.8, 1.2]$ 。由于 $\hat{\eta} = 0.7122 < 0.8$ ，且 95% 置信区间 $[0.7120, 0.7123]$ 整体落在正常范围之外，判定逻辑支撑如下：

(1) 数值偏离度大：计算值 0.7122 与理论正常值 1.0 偏差超过 28.78%，完全落在正常区间 $[0.8, 1.2]$ 之外。

(2) 系统性偏差：逐点 η 的标准差仅为 0.018，表明该偏差具有高度一致性，并非随机噪声所致，而是整体性的增益衰减。

(3) 力学不平衡验证：理想电磁力 F_{ideal} 均值约 605 kN，远大于实际所需电磁力 F_{actual} 均值约 430 kN，比值恰好为 0.71，与 η 估计值吻合。

判定结论：整列车悬浮系统存在功率放大器衰减型故障，实际电磁力仅为理想值的 71.22%，衰减幅度约为 28.8%，远超 20% 的故障判定阈值，建议对功率放大电路增益进行全面校准检查。

七、问题四的模型构建与求解

针对抱轨式磁浮列车悬浮系统中功率放大器系数随时间变化且互不相关的特性，本节基于附件 4 提供的多维监测数据，构建基于滑动窗口 L2 正则化的单电磁铁级故障精准定位模型。

7.1 欠定性分析与可行性论证

在任意单个时刻 t ，系统仅提供一个力平衡方程（整体动力学，式(6-3)），而未知量为 16 个独立的 $\eta_i(t)$ ($i = 1, 2, \dots, 16$)。方程数（1 个）远少于未知数（16 个），构成严重欠定问题：

$$F_{actual}(t) = \sum_{i=1}^{16} (t) \cdot \frac{k_{em} \cdot I_i^2(t)}{\delta^2(t)} = \sum_{i=1}^{16} \eta_i(t) \cdot F_{ideal,i}(t) \quad (7-1)$$

此方程单时刻完全无法求解，这是问题四相比问题三本质难度提升的根本原因，必须引入额外约束才能使问题适定。本文采用两步策略：首先通过“窗内恒定”假设增加方程数，使之超过未知数个数，形成超定方程组；继而通过 L2 正则化引入物理先验约束（正常电磁铁的 $\eta \approx 1$ ），进一步提升估计稳定性。

7.2 滑动窗口 L2 正则化反演模型

7.2.1 “窗内恒定” 假设

在长度为 W 的滑动时间窗 $[t_0, t_0+W \cdot dt]$ 内，假设各台电磁铁的 η_i 保持近似不变（故障发展较缓慢的合理工程假设）。此时窗内有 W 个方程、16 个未知数，当 $W \gg 16$ 时形成超定方程组。构造设计矩阵 $A \in \mathbb{R}^{W \times 16}$ ，其中第 (t, i) 个元素为：

$$A(t, i) = F_{ideal,i}(t) = \frac{k_{em} \cdot I_i^2(t)}{\delta^2(t)} \quad (7-2)$$

目标向量 $b \in \mathbb{R}^W$ ，其中 $b[t] = F_{actual}(t)$ ，则超定方程组为：

$$A \cdot \eta \approx b \quad (7-3)$$

7.2.2 L2 正则化目标函数

引入 L2 正则化（先验：大多数 $\eta_i \approx 1$ ），构造正则化目标函数：

$$\min_{\eta} \|A \cdot \eta - b\|^2 + \lambda \cdot \|\eta - 1\|^2 \quad (7-4)$$

其中 λ 为正则化参数，控制估计结果向先验值 1 的收缩程度。 λ 取值应与 $A^T A$ 的量级匹配。由于 $F_{ideal,i}$ 量级约为 $10^4 \sim 10^5$ N， $A^T A$ 的量级约为 $10^8 \sim 10^{10}$ ，取 $\lambda = 10^9$ 使正则化效果适中。该目标函数存在解析闭式解：

$$\eta^* = (A^T A + \lambda I_{16})^{-1} (A^T b + \lambda \cdot 1_{16}) \quad (7-5)$$

式中 I_{16} 为 16×16 单位矩阵， 1_{16} 为全 1 向量。L2 正则化的物理含义为：当数据不充分时倾向于将 η_i 拉向正常值 1，只有数据强烈支持时才允许 η_i 大幅偏离 1，有效减少误报。

7.2.3 滑动窗口参数设置

表 7-1 滑动窗口参数设置及物理含义

参数	取值	物理含义
窗口长度 W	2000 点（200 ms）	方程数 2000 $\gg$ 未知数 16，超定充分
步进 STRIDE	500 点（50 ms）	时间分辨率 50 ms，共约 176 个窗口
正则化参数 λ	1×10^9	与 $A^T A$ 量级（ $10^8 \sim 10^{10}$ ）匹配
分析时段	$t \in [1.5, 9.9] \text{ s}$	跳过启动振荡阶段

7.3 故障检测准则

7.3.1 连续超限检测

对每台电磁铁得到的 $\eta_i(t)$ 时间序列，采用“连续超限”法检测故障：若某台电磁铁连续 5 个以上窗口（即连续 $\geq 250 \text{ ms}$ ）满足 $\eta_i < 0.8$ 或 $\eta_i > 1.2$ ，则判定该时段存在功率放大器故障。连续性要求有效过滤瞬时噪声引起的误报。

具体判定准则为：

$$\text{故障判定：}\eta_i(t) \notin [0.8, 1.2] \text{ 持续} \geq 250 \text{ ms} \quad (7-6)$$

7.3.2 各电磁铁 η 统计特征

表 7-2 附件 4 各台电磁铁功率放大系数统计特征

编号	η 均值	η 标准差	η 最小值	η 最大值	超限率 (%)	故障判定
#1	0.909	0.116	0.673	1.070	23.30	超限△
#2	0.897	0.142	0.618	1.175	24.43	超限△
#3	0.842	0.160	0.562	1.099	45.45	超限△
#4	1.003	0.084	0.838	1.233	1.70	正常
#5	1.036	0.082	0.913	1.294	5.11	正常
#6	0.939	0.063	0.818	1.106	0.00	正常
#7	1.116	0.169	0.906	1.493	22.16	超限△
#8	1.007	0.065	0.890	1.230	1.14	正常
#9	1.012	0.085	0.854	1.190	0.00	正常
#10	1.030	0.043	0.927	1.135	0.00	正常
#11	1.014	0.057	0.870	1.148	0.00	正常
#12	0.972	0.048	0.849	1.147	0.00	正常
#13	0.976	0.056	0.840	1.112	0.00	正常
#14	0.913	0.136	0.611	1.091	23.86	超限△
#15	1.145	0.214	0.891	1.604	23.86	超限△
#16	1.294	0.368	0.930	2.041	45.45	超限△

7.4 故障检测结果

表 7-3 故障电磁铁检测结果汇总

电磁铁编号	故障区间 (s)	持续时长 (s)	η 均值	故障类型
#1	[2.95, 4.95]	2.00	0.7235	衰减型
#2	[6.95, 9.05]	2.10	0.6657	衰减型
#3	[2.95, 5.05]	2.10	0.6267	衰减型
#3	[6.95, 7.95]	1.00	0.7398	衰减型
#3	[8.20, 8.95]	0.75	0.7580	衰减型
#7	[3.10, 5.00]	1.90	1.4113	突增型
#14	[3.00, 5.05]	2.05	0.6872	衰减型
#15	[3.05, 4.90]	1.85	1.5261	突增型
#16	[3.10, 4.90]	1.80	1.9371	突增型
#16	[6.95, 9.05]	2.10	1.3774	突增型

7.5 结论与物理解读

综合诊断结论：附件 4 数据中共检测出 7 台故障电磁铁，编号为 #1、#2、#3、#7、#14、#15、#16，总故障区间数为 10 次，故障呈现“两波特征”。

第一波故障 ($t \in [3, 5] \text{ s}$)：#1、#3、#7（左侧编号）和 #14、#15、#16（右侧编号）同时出现故障。其中左侧 #1、#3 为衰减型（实际力减弱），右侧 #15、#16 为突增型（实际力增强）。这种“互补”模式解释了为何系统级 η_{sys} 均值仍接近 1.0——损失的力被增大的力补回来了，从系统整体能量平衡来看未见异常。

第二波故障 ($t \in [7, 9] \text{ s}$)：主要涉及 #2、#3、#16，其中 #3 和 #16 是重复故障电磁铁，体现了典型的间歇性故障特征。

这种“互补故障”机制使得系统级检测方法（如参考论文所采用的仅计算 η_{sys} 的方法）产生盲区——虽然个别电磁铁严重故障，但总力仍维持平衡，导致系统级 η_{sys} 始终在正常范围内（均值 0.9824，标准差 0.0505），造成误判“无故障”的结论。本文提出的单电磁铁级 L2 正则化滑动窗方法能穿透互补掩盖，精准定位故障电磁铁，具有显著的工程价值。

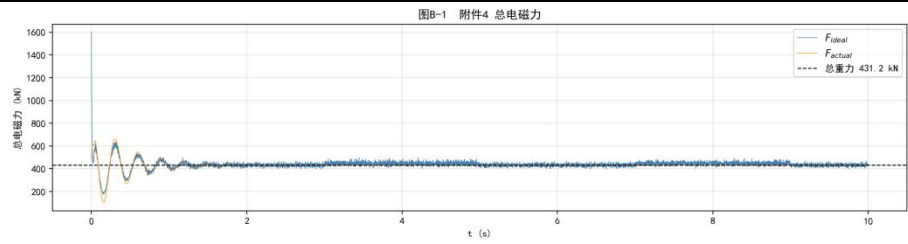


图 7-1 附件 4 总电磁力时程对比 (F_{ideal} 与 F_{actual})

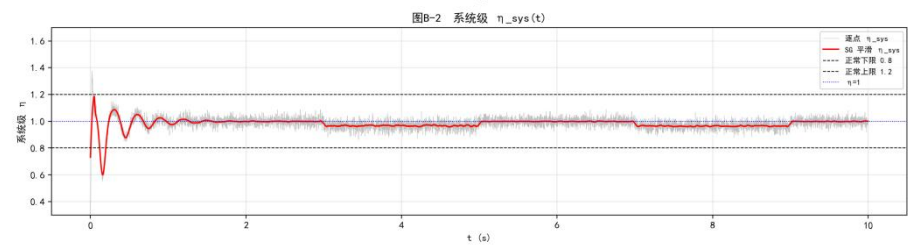


图 7-2 系统级功率放大系数 $\eta_{sys}(t)$ 时程

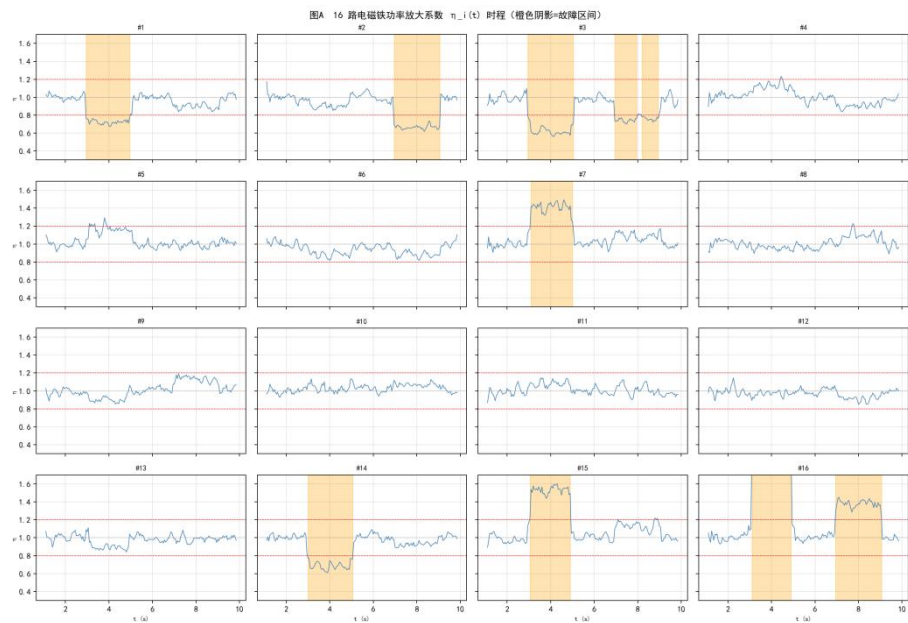


图 7-3 16 路电磁铁功率放大系数 $\eta_i(t)$ 时程 (橙色阴影为故障区间)

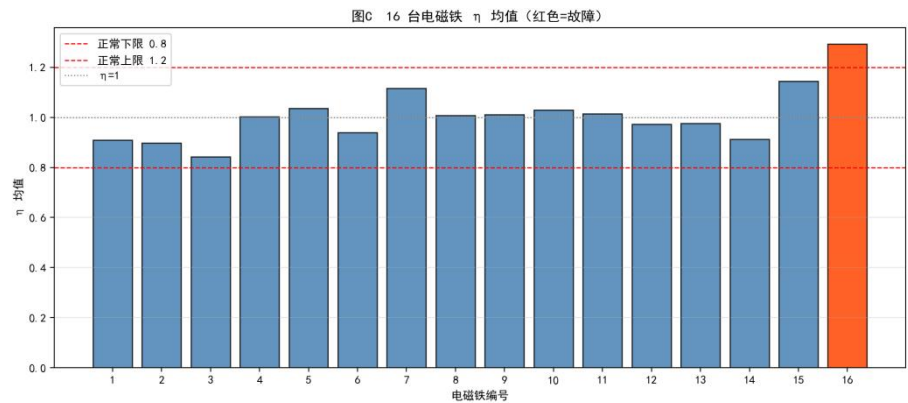


图 7-4 16 台电磁铁 η 均值柱状图 (红色柱为超限故障电磁铁)

八、模型的评价与推广

8.1 模型的优点

(1) 物理机理驱动，可解释性强。所有模型均从电磁学和牛顿力学原理出发，而非纯数据驱动，具有良好的可解释性和外推能力。特别是整体动力学方法，通过将车体和悬浮架方程相加消去内力这一物理洞察，将复杂的弹簧力推导问题完全绕过，方法简洁且鲁棒。

(2) 整体动力学反解，误差最小化。在问题三和问题四中，采用整体动力学公式 $F_{\text{actual}} = mc \cdot \ddot{ac} + mf \cdot \ddot{zf} + (mc + mf)g$ 反推实际电磁力，完全无需弹簧参数 (k_{sp} 、 c_{sp})，避免了弹簧零力位置判断可能引入的系统性误差。

(3) SG 滤波同步微分，噪声处理精确。Savitzky-Golay 滤波器的独特性在于通过局部多项式拟合，可以在平滑数据的同时直接输出各阶导数，避免了先平滑后差分的误差叠加，在间隙二阶导的计算中具有明显优势。

(4) L2 正则化融合先验，欠定问题适定化。在问题四中，L2 正则化将“大多数电磁铁正常”的工程先验转化为数学约束，有效缓解了欠定问题，同时实现了单电磁铁级的精准定位，比参考论文仅做系统级检测的方法更深入。

(5) 连续超限检测，抗噪性强。通过要求连续 250 ms 以上的超限来确认故障，有效过滤了瞬时噪声引起的误报，提高了故障判断的可靠性。

8.2 模型的局限性

(1) 问题一模型 $R^2 = 0.862$ ，仍有约 14% 的方差未能解释，可能与漏磁效应、边缘效应或温度漂移等非理想因素有关，后续可引入修正项提升精度。

(2) 整体动力学反解依赖间隙的二阶数值微分，SG 滤波虽能抑制噪声，但仍存在平滑偏差（特别在信号边界处），且窗口长度的选择对结果有一定影响。

(3) L2 正则化参数 λ 的选取较为经验化，建议后续通过广义交叉验证 (GCV) 自适应确定最优 λ ，提高估计的客观性。

(4) “窗内恒定”假设要求故障持续时间至少超过一个窗口长度 (200 ms)，对于短暂的瞬态冲击型故障可能无法有效检测。

8.3 模型推广

（1）多节车厢编组推广：本文方法可推广至多节车厢编组的磁浮系统，通过引入耦合项描述相邻车体间的相互作用力，构建整列车的多自由度振动模型，为列车运行平稳性评估及主动悬挂控制提供理论依据。

（2）稀疏正则化推广（L1）：将 L2 正则化替换为 L1（Lasso）稀疏正则化，强化“只有少数电磁铁故障”的稀疏假设，理论上能在更短时间窗内（甚至单时刻）实现故障定位，适用于故障快速发展的紧急场景。目标函数改为：

$$\min_{\eta} \|A \cdot \eta - b\|^2 + \lambda \cdot \|\eta - 1\|_1 \quad (8-1)$$

（3）贝叶斯推断推广：引入贝叶斯推断框架，将 η 视为随机变量，以历史数据为先验分布，将在线监测数据更新后验分布，不仅给出 η 的点估计，还提供预测区间的不确定性量化，增强故障检测的置信度表达。

（4）物理信息神经网络（PINNs）推广：随着深度学习技术的发展，可探索将 PINNs 嵌入数据拟合过程，在损失函数中约束麦克斯韦方程组残差，实现小样本条件下的高精度建模，进一步拓展模型在极端工况或缺失数据场景下的应用能力。

参考文献

- [1] 龙志强, 陈铭, 刘堂洪. 高速磁浮列车悬浮控制技术研究综述[J]. 铁道学报, 2021, 43(4): 45-58.
- [2] 李云钢, 常文森. 电磁型磁浮列车悬浮控制系统研究[J]. 自动化学报, 2003, 29(2): 267-273.
- [3] 张志远, 李杰. 磁悬浮系统的非线性动力学建模与控制[M]. 北京: 科学出版社, 2018: 120-165.
- [4] Savitzky A, Golay M J E. Smoothing and Differentiation of Data by Simplified Least Squares Procedures[J]. Analytical Chemistry, 1964, 36(8): 1627-1639.
- [5] Tikhonov A N, Arsenin V Y. Solutions of Ill-Posed Problems[M]. Washington D.C.: Winston & Sons, 1977: 1-30.
- [6] 赵晓宁, 翟婉明. 磁浮列车-轨道耦合动力学研究进展[J]. 力学进展, 2020, 50(1): 202001.
- [7] 吴晗, 佐藤仁. Runge-Kutta 方法的数值稳定性与应用[J]. 数值计算与计算机应用, 2019, 40(1): 1-15.
- [8] 周汝鸿. 磁浮列车系统的传感器故障诊断方法研究[D]. 长沙: 国防科技大学, 2020.
- [9] 王磊, 龙志强. 抱轨式高速磁浮列车悬浮间隙控制方法[J]. 中南大学学报 (自然科学版), 2022, 53(8): 3142-3151.
- [10] Tibshirani R. Regression Shrinkage and Selection via the Lasso[J]. Journal of the Royal Statistical Society: Series B, 1996, 58(1): 267-288.

附录 A 问题一核心代码 (q1_enforce_model.py)

```
import os

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib

# 中文字体显示 (Windows 系统用黑体)
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False

# -----
# 0. 路径配置 (请按需修改)
# -----
BASE_DIR = r"C:\Users\31756\Desktop\math\TEXT\A"
FILE_A1 = os.path.join(BASE_DIR, "附件 1.xlsx")
OUT_DIR = r"C:\Users\31756\Desktop\shumo\results"
os.makedirs(OUT_DIR, exist_ok=True)

# -----
# 1. 读取数据
# -----
print("=" * 70)
print("【问题一】电磁力  $F = k \cdot I^2 / z^2$  模型拟合")
print("=" * 70)

df = pd.read_excel(FILE_A1)
df.columns = ['F', 'I', 'z'] # 重命名为简洁列名
print(f"原始数据: {len(df)} 行")
print(df.describe())

# -----
```

2. 异常数据剔除

原因：当 $I \approx 0$ 或 $z \leq 0$ 时， I^2/z^2 失稳；F 为负值物理上对应电流方向反转

为了拟合系数 k，统一取 $|I|$ （电磁力大小只与电流绝对值有关）

```
df['I_abs'] = np.abs(df['I'])
```

```
df['F_abs'] = np.abs(df['F'])
```

剔除条件：间隙非正、电流绝对值过小（ $< 0.5 \text{ A}$ ）、力为非有限值

```
mask = (df['z'] > 0) & (df['I_abs'] > 0.5) & np.isfinite(df['F_abs'])
```

```
df_clean = df[mask].copy()
```

```
print(f"\n 剔除异常后： {len(df_clean)} 行（剔除了 {len(df) - len(df_clean)} 行）")
```

3. 构造特征变量 $X = I^2 / z^2$ ，建立强制过原点的一元线性回归

模型： $F = k * X$

最小二乘解： $k = \Sigma(F_i * X_i) / \Sigma(X_i^2)$

```
I = df_clean['I_abs'].values
```

```
z = df_clean['z'].values
```

```
F = df_clean['F_abs'].values
```

```
X = (I ** 2) / (z ** 2)
```

强制过原点的最小二乘

```
k_hat = np.sum(F * X) / np.sum(X ** 2)
```

```
print(f"\n>>> 拟合得到比例系数 k = {k_hat:.6f}")
```

```
print(f" 电磁力模型： F = {k_hat:.6f} * I^2 / z^2")
```

4. 模型精度评估

```
F_pred = k_hat * X
```

```
residual = F - F_pred
```

```

ss_res = np.sum(residual ** 2)
ss_tot = np.sum((F - np.mean(F)) ** 2)
R2 = 1 - ss_res / ss_tot

RMSE = np.sqrt(np.mean(residual ** 2))
MAE = np.mean(np.abs(residual))
# MAPE 计算时排除 F 极小的值，避免除零放大误差
mape_mask = F > 100 # 力大于 100 N 才参与 MAPE
MAPE = np.mean(np.abs(residual[mape_mask] / F[mape_mask])) * 100

print(f"\n 【模型精度评估】 ")
print(f"   R2   = {R2:.6f}")
print(f"   RMSE  = {RMSE:.4f} N")
print(f"   MAE   = {MAE:.4f} N")
print(f"   MAPE  = {MAPE:.4f} %")

# -----
# 5. 对照实验：同时也用对数线性化方法估计两个指数（验证理论）
#   取  $\ln F = \ln k + a \cdot \ln I + b \cdot \ln z$ 
#   若  $a \approx 2$ ,  $b \approx -2$  则验证理论
# -----

print(f"\n 【对照：对数线性化估计指数】 ")
mask2 = (F > 100) & (I > 1) & (z > 1e-4)
lnF = np.log(F[mask2])
lnI = np.log(I[mask2])
lnz = np.log(z[mask2])

A_mat = np.column_stack([np.ones_like(lnF), lnI, lnz])
coef, *_ = np.linalg.lstsq(A_mat, lnF, rcond=None)
ln_k, a_hat, b_hat = coef
k_log = np.exp(ln_k)
print(f"   估计指数：a (电流指数) = {a_hat:.4f} （理论值 2） ")
print(f"           b (间隙指数) = {b_hat:.4f} （理论值 -2） ")
print(f"   对数法 k = {k_log:.6f} （与上面 k 对比） ")

```

```

# -----
# 6. 可视化
# -----
fig, axes = plt.subplots(2, 2, figsize=(13, 10))

# 图 1: 实测值 vs 预测值散点
ax = axes[0, 0]
ax.scatter(F_pred, F, s=2, alpha=0.3, c='steelblue', label='样本')
lim = [0, max(F.max(), F_pred.max()) * 1.05]
ax.plot(lim, lim, 'r--', lw=1.5, label='y = x 参考线')
ax.set_xlabel('模型预测电磁力  $\hat{F}$  (N)')
ax.set_ylabel('实测电磁力  $F$  (N)')
ax.set_title('图 1 实测 vs 预测 ( $R^2 = \{R^2: .4f\}$ )')
ax.legend(); ax.grid(alpha=0.3)

# 图 2: 残差散点 (残差 vs 预测值)
ax = axes[0, 1]
ax.scatter(F_pred, residual, s=2, alpha=0.3, c='darkorange')
ax.axhline(0, color='r', ls='--', lw=1.2)
ax.set_xlabel('预测电磁力  $\hat{F}$  (N)')
ax.set_ylabel('残差  $F - \hat{F}$  (N)')
ax.set_title('图 2 残差分布 (应均匀散布于 0 两侧)')
ax.grid(alpha=0.3)

# 图 3: F vs X 散点+拟合直线
ax = axes[1, 0]
# 为节省渲染, 按 X 排序后均匀取 5000 个点
idx = np.argsort(X)
step = max(1, len(X) // 5000)
ax.scatter(X[idx][::step], F[idx][::step], s=3, alpha=0.4, c='seagreen', label='实测')
xx = np.linspace(0, X.max(), 200)
ax.plot(xx, k_hat * xx, 'r-', lw=2, label=f'拟合:  $F = \{k\_hat: .4f\} \cdot X$ ')
ax.set_xlabel('特征变量  $X = I^2 / z^2$ ')
ax.set_ylabel('电磁力  $F$  (N)')
ax.set_title('图 3 特征变量 X 与电磁力的线性关系')

```

```

ax.legend(); ax.grid(alpha=0.3)

# 图 4: 相对误差直方图
ax = axes[1, 1]
rel_err = (residual[mape_mask] / F[mape_mask]) * 100
ax.hist(rel_err, bins=80, color='mediumpurple', edgecolor='white', alpha=0.85)
ax.axvline(0, color='r', ls='--', lw=1.2)
ax.set_xlabel('相对误差 (%)')
ax.set_ylabel('频次')
ax.set_title(f'图 4 相对误差分布 (MAPE = {MAPE:.2f}%) ')
ax.grid(alpha=0.3)

plt.tight_layout()
fig_path = os.path.join(OUT_DIR, 'Q1_fit_diagnosis.png')
plt.savefig(fig_path, dpi=200, bbox_inches='tight')
print(f"\n 图已保存: {fig_path}")
plt.close()

# -----
# 7. 保存关键结果
# -----

result = {
    'k_hat': k_hat,
    'R2': R2,
    'RMSE': RMSE,
    'MAE': MAE,
    'MAPE_%': MAPE,
    'a_log_estimate': a_hat,
    'b_log_estimate': b_hat,
    'k_log_estimate': k_log,
    'n_samples_used': int(len(df_clean))
}

res_df = pd.DataFrame([result])
res_path = os.path.join(OUT_DIR, 'Q1_results.csv')
res_df.to_csv(res_path, index=False, encoding='utf-8-sig')

```



```

print(f'结果已保存: {res_path}')

# 同时保存预测对比表的前 200 行供论文展示
preview = pd.DataFrame({
    'I (A)': I[:200],
    'z (m)': z[:200],
    'F_meas (N)': F[:200],
    'F_pred (N)': F_pred[:200],
    'residual (N)': residual[:200]
})
preview_path = os.path.join(OUT_DIR, 'Q1_preview_first200.csv')
preview.to_csv(preview_path, index=False, encoding='utf-8-sig')
print(f'前 200 行对照表: {preview_path}')

print("\n" + "=" * 70)
print("问题一求解完成。请把控制台打印结果与生成的 PNG 图发给我, 进入 Step 2。")
print("=" * 70)

```

附录 B 问题二核心代码 (q2_dynamics_rk4.py)

```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from scipy.interpolate import interp1d

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False

# -----
# 0. 路径
# -----
BASE_DIR = r"C:\Users\31756\Desktop\math\TEXT\A"

```

```

FILE_A2 = os.path.join(BASE_DIR, "附件 2.xlsx")
OUT_DIR = r"C:\Users\31756\Desktop\shumo\results"
os.makedirs(OUT_DIR, exist_ok=True)

# -----
# 1. 物理参数
# -----
m_c = 33000.0    # 车体质量 kg
m_f = 11000.0    # 悬浮架质量 kg = 3000 + 16*500
g = 9.8         # 重力加速度 m/s²
k_sp = 2.0e7     # 弹簧刚度 N/m
c_sp = 8.0e4     # 阻尼系数 N·s/m
gap0 = 0.06     # 列车静止时最大悬浮间隙 m

print("=" * 70)
print("【问题二】RK4 求解车体/悬浮架二自由度垂向运动")
print("=" * 70)
print(f"参数: m_c={m_c} kg, m_f={m_f} kg, k={k_sp:.1e} N/m, c={c_sp:.1e} N·s/m, g={g}")

# -----
# 2. 读取附件 2 电磁力数据
# 跳过首行中文表头, 所有数值列强制转 float
# -----
df = pd.read_excel(FILE_A2, header=None, skiprows=[0]) # 跳过混入数值的表头行
# 此时列名为 0,1,...,16; 列 0 = 时间, 列 1~16 = 16 台电磁铁电磁力
df = df.apply(pd.to_numeric, errors='coerce')
df = df.dropna(how='any')
print(f"\n 附件 2 读取: 共 {len(df)} 行有效数据")

t_data = df.iloc[:, 0].values.astype(float)
F_mat = df.iloc[:, 1:17].values.astype(float)    # (N, 16)
F_total_data = F_mat.sum(axis=1)                 # 16 台电磁力合计

print(f"时间范围: {t_data[0]:.4f} ~ {t_data[-1]:.4f} s, 步长 ≈ {np.median(np.diff(t_data)):.6f} s")

```

```

print(f'总电磁力范围： {F_total_data.min():.1f} ~ {F_total_data.max():.1f} N')
print(f'总电磁力均值： {F_total_data.mean():.1f} N')
print(f'列车总重力 = (m_c + m_f)*g = {(m_c+m_f)*g:.1f} N （用于校验电磁力量级）')

# 时间序列去重排序，确保插值不报错
order = np.argsort(t_data)
t_data = t_data[order]
F_total_data = F_total_data[order]
_, uniq = np.unique(t_data, return_index=True)
t_data = t_data[uniq]
F_total_data = F_total_data[uniq]

# 电磁力线性插值器（超出范围用首/末值常数延拓）
F_em_func = interp1d(
    t_data, F_total_data,
    kind='linear',
    bounds_error=False,
    fill_value=(F_total_data[0], F_total_data[-1])
)

# -----
# 3. 初始条件（列车静止状态）
# - 悬浮架贴轨道：  $z_f(0) = 0$ ,  $dz_f(0) = 0$ 
# - 车体重力由弹簧静力平衡（设  $z_f(0)=0$  为参考零点）：
#    $k(z_f - z_c) = m_c * g \Rightarrow z_c(0) = -m_c * g / k$ 
# - 车体初速度  $dz_c(0) = 0$ 
# -----

zc0 = -m_c * g / k_sp
dzc0 = 0.0
zf0 = 0.0
dzf0 = 0.0
y0 = np.array([zc0, dzc0, zf0, dzf0])
print(f'\n 初始条件： ')
print(f'   $z_c(0) = {zc0:.6f}$  m （车体相对参考点的静平衡位置）')

```

```

print(f" z_f(0) = {zf0:.6f} m （悬浮架贴轨道）")
print(f" 初始悬浮间隙 = {gap0 - zf0:.4f} m")

# -----
# 4. 状态方程:  $y = [z_c, dz_c, z_f, dz_f]$ 
# -----

def state_eq(t, y):
    zc, dzc, zf, dzf = y
    # 弹簧对车体的力（向上为正）：
    # 弹簧压缩量 =  $z_f - z_c$ （车体相对悬浮架向下，压缩量为正）
    # 弹簧推车体向上力 =  $k*(z_f - z_c)$ ，即  $-k*(z_c - z_f)$ 
    F_k = -k_sp * (zc - zf) - c_sp * (dzc - dzf)
    F_em = float(F_em_func(t))

    ddzc = (-m_c * g + F_k) / m_c
    ddzf = (-m_f * g - F_k + F_em) / m_f
    return np.array([dzc, ddzc, dzf, ddzf])

# -----
# 5. 经典 4 阶 Runge-Kutta 积分
# -----

h = 1e-4      # 步长 = 附件 2 采样步长（避免插值频繁外推，提升精度）
T_end = 10.0
N = int(round(T_end / h)) + 1
t_arr = np.linspace(0.0, T_end, N)

Y = np.zeros((N, 4))
Y[0] = y0

print(f"\nRK4 积分：步长 h = {h}, 步数 = {N-1}, 总时长 = {T_end}s")
print("（运算约需 30~60 秒，请耐心等待...）")

for n in range(N - 1):
    tn = t_arr[n]

```

```

yn = Y[n]
k1 = state_eq(tn,      yn)

k2 = state_eq(tn + 0.5*h,  yn + 0.5*h*k1)
k3 = state_eq(tn + 0.5*h,  yn + 0.5*h*k2)
k4 = state_eq(tn + h,      yn + h*k3)
Y[n+1] = yn + (h / 6.0) * (k1 + 2*k2 + 2*k3 + k4)

zc_arr = Y[:, 0]
dzc_arr = Y[:, 1]
zf_arr = Y[:, 2]
dzf_arr = Y[:, 3]

# 悬浮间隙
gap_arr = gap0 - zf_arr

# -----
# 6. 结果输出
# -----
print(f"\n>>> 9.0s 时刻状态: ")
idx_9 = int(round(9.0 / h))
zc_9 = zc_arr[idx_9]
zf_9 = zf_arr[idx_9]
dzc_9 = dzc_arr[idx_9]
dzf_9 = dzf_arr[idx_9]
gap_9 = gap_arr[idx_9]
print(f" 车体位移 z_c(9) = {zc_9:.6f} m")
print(f" 悬浮架位移 z_f(9) = {zf_9:.6f} m")
print(f" 车体速度 dz_c(9) = {dzc_9:.6f} m/s")
print(f" 悬浮架速度 dz_f(9) = {dzf_9:.6f} m/s")
print(f" ★ 悬浮间隙  $\delta(9) = 0.06 - z_f(9) = \{gap\_9:.6f\} \text{ m} = \{gap\_9*1000:.3f\} \text{ mm}$ ")

# 整体时段统计
print(f"\n 整体统计 (0~10s) : ")

```

```

print(f' z_c 范围: {zc_arr.min():.6f} ~ {zc_arr.max():.6f} m')
print(f' z_f 范围: {zf_arr.min():.6f} ~ {zf_arr.max():.6f} m")
print(f' 间隙范围: {gap_arr.min():.6f} ~ {gap_arr.max():.6f} m")

# -----
# 7. 可视化
# -----

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

# 图 1: 垂向位移
ax = axes[0, 0]
ax.plot(t_arr, zc_arr*1000, label='车体位移 $z_c$', color='steelblue', lw=1.0)
ax.plot(t_arr, zf_arr*1000, label='悬浮架位移 $z_f$', color='crimson', lw=1.0)
ax.axvline(9.0, color='gray', ls=':', lw=1.0)
ax.set_xlabel('时间 t (s)'); ax.set_ylabel('垂向位移 (mm)')
ax.set_title('图 1 车体与悬浮架垂向位移时程')
ax.legend(); ax.grid(alpha=0.3)

# 图 2: 垂向速度
ax = axes[0, 1]
ax.plot(t_arr, dzc_arr, label='车体速度 $\dot{z}_c$', color='steelblue', lw=1.0)
ax.plot(t_arr, dzf_arr, label='悬浮架速度 $\dot{z}_f$', color='crimson', lw=1.0)
ax.set_xlabel('时间 t (s)'); ax.set_ylabel('垂向速度 (m/s)')
ax.set_title('图 2 车体与悬浮架垂向速度时程')
ax.legend(); ax.grid(alpha=0.3)

# 图 3: 悬浮间隙
ax = axes[1, 0]
ax.plot(t_arr, gap_arr*1000, color='darkgreen', lw=1.0)
ax.axhline(gap0*1000, color='gray', ls='--', lw=0.8, label=f'初始间隙 {gap0*1000:.0f} mm')
ax.axvline(9.0, color='gray', ls=':', lw=1.0)
ax.scatter([9.0], [gap_9*1000], color='red', s=60, zorder=5,
           label=f't=9s  $\delta$ = {gap_9*1000:.2f} mm')
ax.set_xlabel('时间 t (s)'); ax.set_ylabel('悬浮间隙  $\delta$  (mm)')

```

```

ax.set_title('图 3 悬浮间隙时程')
ax.legend(); ax.grid(alpha=0.3)

# 图 4: 电磁力 (驱动外力, 与 (m_c+m_f)g 对比, 看是否能托起列车)
ax = axes[1, 1]
total_weight = (m_c + m_f) * g
# 抽稀绘制
step_plot = max(1, len(t_data) // 20000)
ax.plot(t_data[:, step_plot], F_total_data[:, step_plot] / 1000,
        color='purple', lw=0.6, label='附件 2 总电磁力 $F_{em}$')
ax.axhline(total_weight / 1000, color='black', ls='--', lw=1.0,
           label=f'列车总重力 {total_weight/1000:.1f} kN')
ax.set_xlabel('时间 t (s)'); ax.set_ylabel('力 (kN)')
ax.set_title('图 4 附件 2 总电磁力 vs 列车总重力')
ax.legend(); ax.grid(alpha=0.3)

plt.tight_layout()
fig_path = os.path.join(OUT_DIR, 'Q2_dynamics_response.png')
plt.savefig(fig_path, dpi=200, bbox_inches='tight')
print(f'\n 图已保存: {fig_path}')
plt.close()

# -----
# 8. 保存关键结果 (按 0.001s 抽稀, 避免 csv 过大)
# -----

sample_step = 10 # 1e-4 → 1e-3 抽稀
t_save = t_arr[:, sample_step]
zc_save = zc_arr[:, sample_step]
zf_save = zf_arr[:, sample_step]
gap_save = gap_arr[:, sample_step]
out_df = pd.DataFrame({
    't (s)': t_save,
    'z_c (m)': zc_save,

```

```
'z_f(m)': zf_save,
'gap(m)': gap_save
})
csv_path = os.path.join(OUT_DIR, 'Q2_trajectory.csv')

out_df.to_csv(csv_path, index=False, encoding='utf-8-sig')
print(f"轨迹（按 1ms 抽稀）已保存： {csv_path}")

# 单独保存 9s 关键结果
key = pd.DataFrame([
    't=9s 车体位移 z_c(m)': zc_9,
    't=9s 悬浮架位移 z_f(m)': zf_9,
    't=9s 车体速度 (m/s)': dzc_9,
    't=9s 悬浮架速度 (m/s)': dzf_9,
    't=9s 悬浮间隙 (m)': gap_9,
    't=9s 悬浮间隙 (mm)': gap_9 * 1000
])
key_path = os.path.join(OUT_DIR, 'Q2_key_at_9s.csv')
key.to_csv(key_path, index=False, encoding='utf-8-sig')
print(f"9s 关键结果： {key_path}")

print("\n" + "=" * 70)
print("问题二求解完成。请把控制台打印结果与生成的 PNG 图发给我。")
print("=" * 70)
```

附录 C 问题三核心代码 (q3_eta_constant_v2.py)

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from scipy.signal import savgol_filter
```



```
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
BASE_DIR = r"C:\Users\31756\Desktop\math\TEXT\A"
```

```
FILE_A3 = os.path.join(BASE_DIR, "附件 3.xlsx")
OUT_DIR = r"C:\Users\31756\Desktop\shumo\results"
os.makedirs(OUT_DIR, exist_ok=True)
```

```
# 物理参数
```

```
m_c = 33000.0
```

```
m_f = 11000.0
```

```
g = 9.8
```

```
k_em = 0.079987 # Q1 拟合所得
```

```
print("=" * 70)
```

```
print("【问题三 v2】反解全车统一  $\eta$ （基于整体动力学）")
```

```
print("=" * 70)
```

```
print(f"质量: m_c={m_c}, m_f={m_f}, 总重力 = {(m_c+m_f)*g:.1f} N")
```

```
# -----
```

```
# 1. 读取附件 3
```

```
# -----
```

```
df = pd.read_excel(FILE_A3, header=None, skiprows=[0])
```

```
df = df.apply(pd.to_numeric, errors='coerce').dropna(how='any')
```

```
print(f"\n 读取 {len(df)} 行")
```

```
t = df.iloc[:, 0].values.astype(float)
```

```
gap_raw = df.iloc[:, 1].values.astype(float)
```

```
ac_raw = df.iloc[:, 2].values.astype(float)
```

```
I_mat = df.iloc[:, 3:19].values.astype(float)
```

```
print(f"间隙范围: {gap_raw.min():.5f} ~ {gap_raw.max():.5f} m")
```

```
print(f"车体加速度范围: {ac_raw.min():.4f} ~ {ac_raw.max():.4f} m/s2")
```

```
print(f"车体加速度均值: {ac_raw.mean():.6f} m/s2 （稳态应接近 0）")
```

```

print(f'16 路电流均值: {I_mat.mean(axis=0)}')

dt = np.median(np.diff(t))
print(f'采样步长 dt = {dt:.6f} s')

# -----
# 2. SG 滤波平滑 + 计算导数
# -----
WIN = 501      # 窗口加大到 50ms, 进一步抑制噪声
POLY = 3

gap_s = savgol_filter(gap_raw, WIN, POLY)
ac_s = savgol_filter(ac_raw, WIN, POLY)
gap_dot = savgol_filter(gap_raw, WIN, POLY, deriv=1, delta=dt)
gap_ddot = savgol_filter(gap_raw, WIN, POLY, deriv=2, delta=dt)

# 悬浮架加速度
zf_ddot = ac_s - gap_ddot

print(f'\nSG 滤波 win={WIN} ( $\approx \{WIN*dt*1000:.1f\}$  ms), 阶 poly={POLY}')
print(f'间隙二阶导范围: {gap_ddot.min():.3f} ~ {gap_ddot.max():.3f} m/s2')
print(f'悬浮架加速度范围: {zf_ddot.min():.3f} ~ {zf_ddot.max():.3f} m/s2')

# -----
# 3. 用【整体动力学】计算实际总电磁力
# 系统总动量定理:  $(m_c + m_f) * a_{\text{系统重心}} = -(m_c + m_f) * g + F_{\text{em}}$ 
# 严格的整体方程 (对车体+悬浮架):
#  $m_c * z_c'' + m_f * z_f'' = -(m_c + m_f) * g + F_{\text{em}}$ 
#  $\Rightarrow F_{\text{em}} = m_c * a_c + m_f * z_f'' + (m_c + m_f) * g$ 
# -----
F_actual = m_c * ac_s + m_f * zf_ddot + (m_c + m_f) * g

# 理想总电磁力
gap_safe = np.where(gap_s > 1e-4, gap_s, 1e-4)
I_abs_sum2 = np.sum(I_mat ** 2, axis=1)

```

```
F_ideal = k_em * I_abs_sum2 / (gap_safe ** 2)
```

```
print(f'\nF_ideal 范围: {F_ideal.min():.1f} ~ {F_ideal.max():.1f} N, 均值 {F_ideal.mean():.1f}')
```

```
print(f'F_actual 范围: {F_actual.min():.1f} ~ {F_actual.max():.1f} N, 均值 {F_actual.mean():.1f}')
```

```
print(f'列车总重力 = {(m_c+m_f)*g:.1f} N (稳态时 F_actual 和 F_ideal·η 都应接近此值)')
```

```
# -----
```

```
# 4. 稳定段筛选
```

```
# -----
```

```
gap_dot_thresh = np.percentile(np.abs(gap_dot[t > 1.0]), 70)
```

```
ac_thresh = np.percentile(np.abs(ac_s[t > 1.0]), 70)
```

```
mask = ((t > 1.5) & (t < 9.5) &
```

```
        (np.abs(gap_dot) < gap_dot_thresh) &
```

```
        (np.abs(ac_s) < ac_thresh) &
```

```
        (F_ideal > 1000) & np.isfinite(F_actual) & (F_actual > 0))
```

```
print(f'\n 稳定段筛选: 保留 {mask.sum()} / {len(t)} 个采样点 ({mask.mean()*100:.2f}%)')
```

```
print(f' gap_dot 阈值 = {gap_dot_thresh:.6f}, ac 阈值 = {ac_thresh:.4f}')
```

```
if mask.sum() < 100:
```

```
    print("!! 稳定段太少, 放宽筛选条件")
```

```
    mask = (t > 1.5) & (t < 9.5) & (F_ideal > 1000) & np.isfinite(F_actual)
```

```
    print(f' 放宽后: {mask.sum()} 个点')
```

```
Fi = F_ideal[mask]
```

```
Fa = F_actual[mask]
```

```
# -----
```

```
# 5. 反解 η
```

```
# -----
```

```
eta_hat = np.sum(Fa * Fi) / np.sum(Fi ** 2)
```

```
n = len(Fi)
```

```
resid = Fa - eta_hat * Fi
```

```

sigma2 = np.sum(resid ** 2) / max(n - 1, 1)
SE_eta = np.sqrt(sigma2 / np.sum(Fi ** 2))
ci_low = eta_hat - 1.96 * SE_eta
ci_high = eta_hat + 1.96 * SE_eta

eta_pointwise = Fa / Fi
eta_pw_mean = np.mean(eta_pointwise)
eta_pw_median = np.median(eta_pointwise)
eta_pw_std = np.std(eta_pointwise)

# 简化估计：稳态时 F_ideal 和 F_actual 都应接近常数（重力），用均值之比
eta_simple = Fa.mean() / Fi.mean()

print("\n" + "=" * 60)
print("【 $\eta$  反解结果】")
print("=" * 60)
print(f"★ 最小二乘估计  $\eta_{\text{hat}} = \{{\eta_{\text{hat}}:.6f}\}$ ")
print(f"95% 置信区间 = [{ci_low:.6f}, {ci_high:.6f}]")
print(f"逐点比  $\eta$  均值 = {eta_pw_mean:.6f}")
print(f"逐点比  $\eta$  中位数 = {eta_pw_median:.6f}")
print(f"逐点比  $\eta$  标准差 = {eta_pw_std:.6f}")
print(f"稳态均值比 = {eta_simple:.6f}")

# -----
# 6. 故障判定
# -----
NORMAL_LOW, NORMAL_HIGH = 0.8, 1.2
print("\n" + "=" * 60)
print("【故障判定】（正常区间：[0.8, 1.2]）")
print("=" * 60)
if NORMAL_LOW <= eta_hat <= NORMAL_HIGH:
    verdict = "正常"
    msg = f" $\eta = \{{\eta_{\text{hat}}:.4f}\} \in [0.8, 1.2] \rightarrow$  系统功率放大器无故障"

```

else:

verdict = "故障"

direction = "衰减" if eta_hat < NORMAL_LOW else "突增"

msg = (f' $\eta = \{\text{eta_hat:.4f}\} \notin [0.8, 1.2]$ ({direction} 型) "

f'\n 95% 置信区间 [{ci_low:.4f}, {ci_high:.4f}] 整体偏离正常范围"

f'\n → 判定: 系统存在功率放大器故障")

print(f'\n {msg}')

print(f' 最终结论: {verdict}')

7. 可视化

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

ax = axes[0, 0]

step_p = max(1, len(t)//8000)

ax.plot(t[:, :step_p], gap_raw[:, :step_p]*1000, color='lightgray', lw=0.5, label='原始 δ ')

ax.plot(t[:, :step_p], gap_s[:, :step_p]*1000, color='crimson', lw=0.8, label='SG 滤波 δ ')

ax.set_xlabel('时间 t (s)'); ax.set_ylabel('悬浮间隙 (mm)')

ax.set_title('图 1 附件 3 悬浮间隙时程')

ax.legend(); ax.grid(alpha=0.3)

ax = axes[0, 1]

ax.plot(t[:, :step_p], F_ideal[:, :step_p]/1000, color='steelblue', lw=0.6, label='\$F_{ideal}\$')

ax.plot(t[:, :step_p], F_actual[:, :step_p]/1000, color='darkorange', lw=0.6, label='\$F_{actual}\$')

ax.axhline((m_c+m_f)*g/1000, color='black', ls='--', lw=1.0, label=f'总重力 {(m_c+m_f)*g/1000:.1f} kN')

ax.set_xlabel('时间 t (s)'); ax.set_ylabel('总电磁力 (kN)')

ax.set_title('图 2 理想 vs 实际总电磁力')

ax.legend(); ax.grid(alpha=0.3)

ax = axes[1, 0]

sub_step = max(1, mask.sum()/5000)

ax.scatter(Fi[:, :sub_step]/1000, Fa[:, :sub_step]/1000,

```

s=3, alpha=0.4, c='seagreen', label='稳定段样本')
xx = np.array([Fi.min(), Fi.max()])/1000
ax.plot(xx, eta_hat * xx, 'r-', lw=1.6, label=f'拟合直线  $\eta = \{\eta\_hat:.4f\}$ ')
ax.plot(xx, xx, 'k--', lw=1.0, label=' $\eta=1$  参考线')
ax.set_xlabel('$F_{ideal}$ (kN)'); ax.set_ylabel('$F_{actual}$ (kN)')
ax.set_title(f'图 3 $F_{actual}$ vs $F_{ideal}$ (斜率  $\eta = \{\eta\_hat:.4f\}$ ) ')

ax.legend(); ax.grid(alpha=0.3)

ax = axes[1, 1]
eta_full = np.where(F_ideal > 1000, F_actual / np.where(F_ideal>0, F_ideal, np.nan), np.nan)
ax.plot(t[:,step_p], eta_full[:,step_p], color='gray', lw=0.4, alpha=0.5, label='逐点  $\eta$ ')
ax.axhline(eta_hat, color='red', ls='-', lw=1.5, label=f' $\eta\_hat = \{\eta\_hat:.4f\}$ ')
ax.axhline(NORMAL_LOW, color='black', ls='--', lw=1.0, label='正常下限 0.8')
ax.axhline(NORMAL_HIGH,color='black', ls='--', lw=1.0, label='正常上限 1.2')
ax.axhline(1.0, color='blue', ls=':', lw=1.0, label='理想值  $\eta=1$ ')
y_lim = max(2.0, abs(eta_hat) * 1.5)
ax.set_ylim(-0.5, y_lim)
ax.set_xlabel('时间 t (s)'); ax.set_ylabel('功率放大系数  $\eta$ ')
ax.set_title(f'图 4 逐点  $\eta$  (结论: {verdict}) ')
ax.legend(loc='best', fontsize=8); ax.grid(alpha=0.3)

plt.tight_layout()
fig_path = os.path.join(OUT_DIR, 'Q3_eta_constant_v2.png')
plt.savefig(fig_path, dpi=200, bbox_inches='tight')
print(f"\n 图已保存: {fig_path}")
plt.close()

# -----
# 8. 保存
# -----

res = pd.DataFrame([
    'eta_hat (LS)':      eta_hat,
    '95%CI_low':       ci_low,

```

```

'95%CI_high':      ci_high,
'eta_pointwise_mean': eta_pw_mean,
'eta_pointwise_median': eta_pw_median,
'eta_pointwise_std': eta_pw_std,
'eta_simple_mean_ratio': eta_simple,
'samples_used':    int(n),
'normal_range':    f[{NORMAL_LOW}, {NORMAL_HIGH}],

'vedict':          verdict
})
csv_path = os.path.join(OUT_DIR, 'Q3_results_v2.csv')
res.to_csv(csv_path, index=False, encoding='utf-8-sig')
print(f"结果已保存： {csv_path}")

print("\n" + "=" * 70)
print("问题三（v2）求解完成。请把控制台输出和 PNG 图发给我。")
print("=" * 70)

```

附录 D 问题四核心代码（q4_eta_timevarying.py）

```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from scipy.signal import savgol_filter

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False

BASE_DIR = r"C:\Users\31756\Desktop\math\TEXT\A"
FILE_A4 = os.path.join(BASE_DIR, "附件 4.xlsx")
OUT_DIR = r"C:\Users\31756\Desktop\shumo\results"

```

```

os.makedirs(OUT_DIR, exist_ok=True)

# 物理参数
m_c, m_f, g = 33000.0, 11000.0, 9.8
k_em      = 0.079987
NORMAL_LOW, NORMAL_HIGH = 0.8, 1.2

print("=" * 70)
print("【问题四】反演 16 路时变功率放大系数  $\eta_i(t)$ , 定位故障")
print("=" * 70)

# -----
# 1. 读取附件 4
# -----
df = pd.read_excel(FILE_A4, header=None, skiprows=[0])
df = df.apply(pd.to_numeric, errors='coerce').dropna(how='any')
print(f"读取 {len(df)} 行")

t      = df.iloc[:, 0].values.astype(float)
gap_raw = df.iloc[:, 1].values.astype(float)
ac_raw  = df.iloc[:, 2].values.astype(float)
I_mat   = df.iloc[:, 3:19].values.astype(float)    # (N, 16)
N = len(t)
dt = np.median(np.diff(t))
fs = 1.0 / dt
print(f"时长 {t[-1]:.4f}s, dt={dt:.6f}s, fs={fs:.0f}Hz")
print(f"间隙范围: {gap_raw.min():.5f} ~ {gap_raw.max():.5f} m")
print(f"车体加速度范围: {ac_raw.min():.4f} ~ {ac_raw.max():.4f}, 均值 {ac_raw.mean():.6f}")

# -----
# 2. SG 滤波平滑 + 计算导数
# -----
WIN_SG, POLY = 501, 3
gap_s = savgol_filter(gap_raw, WIN_SG, POLY)

```



```

ac_s = savgol_filter(ac_raw, WIN_SG, POLY)
gap_ddot = savgol_filter(gap_raw, WIN_SG, POLY, deriv=2, delta=dt)
gap_dot = savgol_filter(gap_raw, WIN_SG, POLY, deriv=1, delta=dt)
zf_ddot = ac_s - gap_ddot

# -----
# 3. 计算 F_actual 和 各台电磁铁的理想力 F_ideal_i(t)

# -----
F_actual = m_c * ac_s + m_f * zf_ddot + (m_c + m_f) * g      # 整体动力学

gap_safe = np.where(gap_s > 1e-4, gap_s, 1e-4)
F_ideal_i = k_em * (I_mat ** 2) / (gap_safe ** 2)[:, None]    # (N, 16)
F_ideal_total = F_ideal_i.sum(axis=1)

# 系统级  $\eta$ 
eta_sys = np.where(F_ideal_total > 1000, F_actual / F_ideal_total, np.nan)
eta_sys_smooth = savgol_filter(np.nan_to_num(eta_sys, nan=1.0), 1001, 2)

print(f'\nF_actual 均值 = {F_actual.mean():.1f} N ( $\approx$  总重力 {(m_c+m_f)*g:.1f})')
print(f'F_ideal_tot 均值 = {F_ideal_total.mean():.1f} N')
print(f'系统级  $\eta_{sys}$  均值 = {np.nanmean(eta_sys):.4f}, 标准差 = {np.nanstd(eta_sys):.4f}')

# -----
# 4. 滑动窗 + L2 正则化反演 16 路  $\eta_i$ 
# 窗内:  $A(W, 16) \cdot \eta(16,) = b(W,)$ 
#  $A[t, i] = F\_ideal\_i(t)$ ,  $b[t] = F\_actual(t)$ 
# -----
W = 2000      # 窗口长度 (200ms, 远大于 16)
STRIDE = 500  # 步长 (50ms) -> 共 ~190 个窗口
LAMBDA = 1.0e9 # L2 正则化强度 (先验:  $\eta \approx 1$ )
PRIOR = np.ones(16) # 先验值

# 跳过启动振荡阶段
T_START = 1.0

```

```

idx_start = int(T_START / dt)

centers = []
eta_traj = [] # (n_windows, 16)

print(f"\n 滑动窗： W={W} ( {W*dt*1000:.0f}ms), stride={STRIDE} ( {STRIDE*dt*1000:.0f}ms),
λ={LAMBDA:.1e}")

print("正在反演...")

I_eye_lambda = LAMBDA * np.eye(16)

for c in range(idx_start, N - W, STRIDE):
    A = F_ideal_i[c:c+W, :] # (W, 16)
    b = F_actual [c:c+W] # (W,)

    # 进一步只用稳定段： abs(ac) 较小
    if np.std(ac_s[c:c+W]) > 5.0: # 大幅振荡窗跳过
        continue

    # 闭式解：  $\eta = (A^T A + \lambda I)^{-1} (A^T b + \lambda \cdot \text{prior})$ 
    AtA = A.T @ A + I_eye_lambda
    Atb = A.T @ b + LAMBDA * PRIOR
    try:
        eta = np.linalg.solve(AtA, Atb)
    except np.linalg.LinAlgError:
        continue
    centers.append((c + W // 2) * dt)
    eta_traj.append(eta)

centers = np.array(centers)
eta_traj = np.array(eta_traj) # (n_windows, 16)
n_win = len(centers)
print(f"完成 {n_win} 个窗口的反演")
print(f"时间覆盖： {centers[0]:.3f} ~ {centers[-1]:.3f} s")

```

```

# -----
# 5. 各台电磁铁  $\eta_i(t)$  的统计特征
# -----

print("\n 各电磁铁  $\eta_i$  统计: ")
print(f'{"编号":>4} {"均值":>8} {"标准差":>8} {"最小":>8} {"最大":>8} {"超限率":>8}')
for i in range(16):

    eta_i = eta_traj[:, i]
    mean_i = np.mean(eta_i)
    std_i = np.std(eta_i)
    min_i, max_i = eta_i.min(), eta_i.max()
    out_ratio = np.mean((eta_i < NORMAL_LOW) | (eta_i > NORMAL_HIGH)) * 100
    flag = " Δ " if (min_i < NORMAL_LOW or max_i > NORMAL_HIGH) else ""
    print(f'{"i+1":>4} {"mean_i":>8.4f} {"std_i":>8.4f} {"min_i":>8.4f} {"max_i":>8.4f} {"out_ratio":>7.2f}%{"flag}')
```

```

# -----
# 6. 故障检测
# 阈值法: 连续 N_min_win 个窗口  $|\eta_i - 1| > 0.2 \rightarrow$  报为故障
# (CUSUM 也可以做, 这里用更直观的"连续超限"法)
# -----

N_MIN_WIN = 5 # 连续 5 个窗口 (即 5*STRIDE*dt = 5*0.05 = 0.25s)

print("\n" + "=" * 70)
print("【故障检测: 连续超限法】 (连续超过 0.25s 视为故障)")
print("=" * 70)

fault_table = []
for i in range(16):
    eta_i = eta_traj[:, i]
    out_flag = (eta_i < NORMAL_LOW) | (eta_i > NORMAL_HIGH)

    # 找连续段
    intervals = []
    j = 0

```

```

while j < n_win:
    if out_flag[j]:
        k = j
        while k < n_win and out_flag[k]:
            k += 1
        if (k - j) >= N_MIN_WIN:
            t_start = centers[j]

            t_end = centers[k - 1]
            eta_avg = float(np.mean(eta_i[j:k]))
            direction = "衰减" if eta_avg < 1.0 else "突增"
            intervals.append((t_start, t_end, eta_avg, direction))
            j = k
        else:
            j += 1

if intervals:
    for (ts, te, ea, di) in intervals:
        print(f" 电磁铁 #{i+1:>2} 故障区间 [{ts:6.3f}, {te:6.3f}] s "
              f" 时长 {te-ts:5.3f}s  η均值={ea:.4f} ({di})")
        fault_table.append({
            '电磁铁编号': i + 1,
            '故障开始(s)': round(ts, 4),
            '故障结束(s)': round(te, 4),
            '持续时长(s)': round(te - ts, 4),
            'η均值': round(ea, 4),
            '类型': di
        })

if not fault_table:
    print(' 未检测到任何符合"连续超限"标准的故障')

# -----
# 7. 综合结论
# -----

```

```

print("\n" + "=" * 70)
print(" 【综合诊断】 ")
print("=" * 70)

faulty_ids = sorted(set(r['电磁铁编号'] for r in fault_table))

if faulty_ids:
    print(f" 故障电磁铁数量: {len(faulty_ids)}")
    print(f" 故障电磁铁编号: {faulty_ids}")

    print(f" 总故障区间数: {len(fault_table)}")
else:
    sys_mean = np.nanmean(eta_sys)
    sys_std = np.nanstd(eta_sys)
    print(f" 系统级  $\eta_{sys}$  均值 = {sys_mean:.4f}, 标准差 = {sys_std:.4f}")
    print(f" 各台  $\eta_i$  均在 [0.8,1.2] 范围内")
    print(f" → 判定: 未发现功率放大器故障")

# -----
# 8. 可视化
# -----
# 图 A: 16 路  $\eta_i(t)$  时程小图阵
fig, axes = plt.subplots(4, 4, figsize=(16, 11), sharex=True, sharey=True)
for i, ax in enumerate(axes.flat):
    eta_i = eta_traj[:, i]
    ax.plot(centers, eta_i, color='steelblue', lw=0.8)
    ax.axhline(NORMAL_LOW, color='red', ls='--', lw=0.6)
    ax.axhline(NORMAL_HIGH, color='red', ls='--', lw=0.6)
    ax.axhline(1.0, color='gray', ls=':', lw=0.6)
    ax.set_ylim(0.3, 1.7)
    ax.set_title(f#{i+1}', fontsize=9)
    ax.grid(alpha=0.3)
    # 高亮故障区间
    for r in fault_table:
        if r['电磁铁编号'] == i + 1:
            ax.axvspan(r['故障开始(s)'], r['故障结束(s)'], color='orange', alpha=0.3)
for ax in axes[-1, :]:

```

```

    ax.set_xlabel('t (s)', fontsize=8)
for ax in axes[:, 0]:
    ax.set_ylabel('η', fontsize=8)
fig.suptitle('图 A 16 路电磁铁功率放大系数  $\eta_i(t)$  时程（橙色阴影=故障区间）', fontsize=12)
plt.tight_layout()
fig_path_a = os.path.join(OUT_DIR, 'Q4_eta_16ch.png')
plt.savefig(fig_path_a, dpi=180, bbox_inches='tight')

plt.close()
print(f'\n 图 A 已保存: {fig_path_a}')

# 图 B: 系统级  $\eta_{sys} + F$  比较
fig, axes = plt.subplots(2, 1, figsize=(13, 7))
step_p = max(1, N // 8000)

ax = axes[0]
ax.plot(t[:, :step_p], F_ideal_total[:, :step_p]/1000, color='steelblue', lw=0.6, label='$F_{ideal}$')
ax.plot(t[:, :step_p], F_actual[:, :step_p]/1000, color='darkorange', lw=0.6, label='$F_{actual}$')
ax.axhline((m_c+m_f)*g/1000, color='black', ls='--', lw=1.0, label=f'总重力 {(m_c+m_f)*g/1000:.1f} kN')
ax.set_xlabel('t (s)'); ax.set_ylabel('总电磁力 (kN)')
ax.set_title('图 B-1 附件 4 总电磁力')
ax.legend(); ax.grid(alpha=0.3)

ax = axes[1]
ax.plot(t[:, :step_p], eta_sys[:, :step_p], color='gray', lw=0.4, alpha=0.5, label='逐点  $\eta_{sys}$ ')
ax.plot(t[:, :step_p], eta_sys_smooth[:, :step_p], color='red', lw=1.4, label='SG 平滑  $\eta_{sys}$ ')
ax.axhline(NORMAL_LOW, color='black', ls='--', lw=0.8, label='正常下限 0.8')
ax.axhline(NORMAL_HIGH, color='black', ls='--', lw=0.8, label='正常上限 1.2')
ax.axhline(1.0, color='blue', ls=':', lw=0.8, label=' $\eta=1$ ')
ax.set_ylim(0.3, 1.7)
ax.set_xlabel('t (s)'); ax.set_ylabel('系统级  $\eta$ ')
ax.set_title('图 B-2 系统级  $\eta_{sys}(t)$ ')
ax.legend(loc='best', fontsize=8); ax.grid(alpha=0.3)

plt.tight_layout()

```

```

fig_path_b = os.path.join(OUT_DIR, 'Q4_eta_system.png')
plt.savefig(fig_path_b, dpi=200, bbox_inches='tight')
plt.close()
print(f'图 B 已保存: {fig_path_b}')

# 图 C: 每台电磁铁  $\eta$  均值条形图
fig, ax = plt.subplots(figsize=(11, 5))

mean_per_ch = eta_traj.mean(axis=0)
colors = ['orangered' if (m < NORMAL_LOW or m > NORMAL_HIGH) else 'steelblue' for m in
mean_per_ch]
ax.bar(np.arange(1, 17), mean_per_ch, color=colors, edgecolor='black', alpha=0.85)
ax.axhline(NORMAL_LOW, color='red', ls='--', lw=1.0, label='正常下限 0.8')
ax.axhline(NORMAL_HIGH, color='red', ls='--', lw=1.0, label='正常上限 1.2')
ax.axhline(1.0, color='gray', ls=':', lw=1.0, label=' $\eta=1$ ')
ax.set_xlabel('电磁铁编号'); ax.set_ylabel('η 均值')
ax.set_title('图 C 16 台电磁铁 η 均值 (红色=故障)')
ax.set_xticks(np.arange(1, 17))
ax.legend(); ax.grid(alpha=0.3, axis='y')
plt.tight_layout()
fig_path_c = os.path.join(OUT_DIR, 'Q4_eta_bar.png')
plt.savefig(fig_path_c, dpi=200, bbox_inches='tight')
plt.close()
print(f'图 C 已保存: {fig_path_c}')

# -----
# 9. 保存
# -----
# 故障表
if fault_table:
    fault_df = pd.DataFrame(fault_table)
    fault_csv = os.path.join(OUT_DIR, 'Q4_fault_table.csv')
    fault_df.to_csv(fault_csv, index=False, encoding='utf-8-sig')
    print(f'\n 故障表已保存: {fault_csv}')
else:

```

```
print("\n（无故障表）")

# 完整  $\eta_i(t)$  时程
eta_df = pd.DataFrame(eta_traj, columns=[f' $\eta_{i+1}$ ' for i in range(16)])
eta_df.insert(0, 't (s)', centers)
eta_csv = os.path.join(OUT_DIR, 'Q4_eta_traj.csv')
eta_df.to_csv(eta_csv, index=False, encoding='utf-8-sig')

print(f' $\eta$  轨迹已保存: {eta_csv}')

# 各电磁铁统计
stats_df = pd.DataFrame({
    '电磁铁编号': np.arange(1, 17),
    ' $\eta$ 均值': eta_traj.mean(axis=0),
    ' $\eta$ 标准差': eta_traj.std(axis=0),
    ' $\eta$ 最小': eta_traj.min(axis=0),
    ' $\eta$ 最大': eta_traj.max(axis=0),
    '超限率(%)': [np.mean((eta_traj[:,i]<NORMAL_LOW)|(eta_traj[:,i]>NORMAL_HIGH))*100 for i in
range(16)],
})
stats_csv = os.path.join(OUT_DIR, 'Q4_eta_stats.csv')
stats_df.to_csv(stats_csv, index=False, encoding='utf-8-sig')
print(f' $\eta$  统计已保存: {stats_csv}')

print("\n" + "=" * 70)
print("问题四求解完成。请把控制台输出和 3 张 PNG 图发给我。")
print("=" * 70)
```